线性表

阅读顺序

点标题跳到对应页

线性表与操作 第 5 页
把位序、下标和头结点分开理解。

顺序表 第 38 页
用连续地址定位元素，插入时从后向前搬移。

单链表 第 58 页
跟踪头指针、前驱和当前结点的变化。

有序单链表合并 第 85 页
同时跟踪两条输入链和结果链的尾结点。

静态链表 第 117 页
用数组下标连接结点，理解备用链的作用。

一元多项式相加 第 143 页
按指数归并，处理同次项相消和剩余项。

易错点

已知待插位置的前驱时，单链表接入结点是常数时间。若要先按位序找前驱，还需遍历。

第 87 页的三种比较结果应互斥。移动结点指针后，下一轮再比较。

随堂自测

长为 5 的顺序表在第 2 位之前插入，需要移动几个元素？

答案 4 个，原第 2 至第 5 位都要后移。

第二章 线性表

无法显示该图片。

- 线性表是一种最基本的数据结构，它不仅自己有广泛的应用，也是其它一些数据结构的基础，后面几章即将讨论的栈、队、串等就是线性表的特例。
- 本章先讨论线性表的逻辑结构与抽象操作，接着给出线性表的两种存贮映射方法，最后以一个经典的问题——一元n次多项式相加说明线性表的应用。
- 本章虽讨论的是线性表,但涉及的许多问题（如存贮映射的实现、抽象操作的表示与应用方式等）带有一定的普遍性。

线性表是一种最简单的**线性结构**

线性结构的基本特征为:

线性结构是一个数据元素的有序（次序）集

1．集合中必存在唯一的一个“第一元素”；

2．集合中必存在唯一的一个“最后元素”；

3．除最后元素在外，均有唯一的后继；

4．除第一元素之外，均有唯一的前驱。

2.1 线性表的类型定义

2.2 线性表类型的实现
—— 顺序映象

2.3 线性表类型的实现
—— 链式映象

2.4 一元多项式的表示



2.1

线性表的类型定义

抽象数据类型**线性表**的定义如下：

ADT List {

数据对象：

$D=\{\ a_i \mid a_i \in ElemSet,\ i=1,2,...,n,\ \ n\geq 0\ \}$

{称 **n** 为线性表的**表长**；

称 **n=0** 时的线性表为**空表**。}

数据关系：

$R1=\{\ <a_{i-1},a_i>\ |a_{i-1},a_i\in D,\ \ i=2,...,n\ \}$

{设线性表为 $(a_1,\ a_2,\ \ldots,\ a_i,\ \ldots,\ a_n)$，

称 i 为 a_i 在线性表中的**位序**。}

基本操作:

结构初始化操作

结构销毁操作

引用型操作

加工型操作

} ADT List



初始化操作

InitList(&L)

操作结果：

构造一个空的线性表L。

结构销毁操作

DestroyList(&L)

初始条件：　线性表 L 已存在。

操作结果：　销毁线性表 L。

引用型操作：

ListEmpty(L)

ListLength(L)

PriorElem(L, cur_e, &pre_e)

NextElem(L, cur_e, &next_e)

GetElem(L, i, &e)

LocateElem(L, e, compare( ))

ListTraverse(L, visit( ))

ListEmpty(L)
(线性表判空)

初始条件：线性表L已存在。

操作结果：若L为空表，则返回TRUE，否则FALSE。

ListLength(L)

(求线性表的长度)

初始条件：线性表L已存在。

操作结果：返回L中元素个数。



PriorElem(L, cur_e, &pre_e)

（求数据元素的前驱）

初始条件：　线性表L已存在。

操作结果：　若cur_e是L的元素，但不是第一个，则用pre_e 返回它的前驱，否则操作失败，pre_e无定义。

NextElem(L, cur_e, &next_e)

（求数据元素的后继）

初始条件：线性表L已存在。

操作结果：若cur_e是L的元素，但不是最后一个，则用next_e返回它的后继，否则操作失败，next_e无定义。

GetElem(L, i, &e)

（求线性表中某个数据元素）

初始条件：线性表L已存在，
且 1≤i≤LengthList(L)。

操作结果：用 e 返回L中第 i 个元素的值。

LocateElem(L, e, compare( ))

（定位函数）

初始条件：线性表L已存在，e为给定值，compare( )是元素判定函数。

操作结果：返回L中第1个与e满足关系compare( )的元素的位序。若这样的元素不存在，则返回值为0。

ListTraverse(L, visit( ))

(遍历线性表)

初始条件：线性表L已存在，
Visit() 为某个访问函数。

操作结果：**依次**对L的每个元素调用函数visit( )。一旦visit( )失败，则操作失败。

加工型操作

ClearList(&L)

PutElem(&L, i, &e)

ListInsert(&L, i, e)

ListDelete(&L, i, &e)



ClearList(&L)

（线性表置空）

初始条件：　线性表L已存在。

操作结果：　将L重置为空表。

PutElem(&L, i, &e)

（改变数据元素的值）

初始条件：线性表L已存在，
且 1≤i≤LengthList(L) 。

操作结果：L中第i个元素赋值同e的值。

ListInsert(&L, i, e)

（插入数据元素）

初始条件：线性表L已存在，
且 1≤i≤LengthList(L)+1 。

操作结果：在L的第i个元素之前**插入**
新的元素e，L的长度增1。

ListDelete(&L, i, &e)

(删除数据元素)

初始条件：线性表L已存在且非空，
1≤i≤LengthList(L) 。

操作结果：删除L的第i个元素，并用e返回其值，L的长度减1。

利用上述定义的**线性表**

可以实现其它更复杂的操作

例 2-1

例 2-2

例 2-3

例 2-1

假设:有两个**集合 A 和 B** 分别用两个**线性表 LA 和 LB** 表示，即：线性表中的数据元素即为集合中的成员。

现要求一个新的集合A = A∪B。

上述问题可演绎为：

要求对线性表作如下操作：

扩大线性表 LA，将**存在于线性表 LB 中**而**不存在于线性表 LA 中**的数据元素**插入到线性表 LA 中**去。

操作步骤：

1．从线性表LB中依次察看每个数据元素;

GetElem(LB, i)→e

2．依值在线性表LA中进行查访;

LocateElem(LA, e, equal( ))

3．若不存在，则插入之。

ListInsert(LA, n+1, e)

```
void union(List &La, List Lb) {
    La_len = ListLength(La);
    Lb_len = ListLength(Lb);
for (i = 1;  i <= Lb_len;  i++) {
GetElem(Lb, i, e); // 取Lb中第i个数据元素赋给e
if (!LocateElem(La, e, equal( )) )
   ListInsert(La, ++La_len, e); // 缺失元素追加到表尾
    // La中不存在和 e 相同的数据元素，则插入之
}
} // union
```

例 2-2

已知一个非纯集合 B，试构造一个纯集合 A，使 A中只包含 B 中所有值各不相 同的数据元素。

仍选用**线性表**表示集合。



集合 B



集合 A

从集合 B 取出物件放入集合 A

要求集合A中**同样物件不能有两件以上**

因此，**算法的策略应该和例2-1相同**

```
void union(List &La, List Lb) {
   InitList(La);   // 构造(空的)线性表LA
   La_len=ListLength(La);
   Lb_len=ListLength(Lb);
   for (i = 1;  i <= Lb_len;  i++) {
   GetElem(Lb, i, e); // 取Lb中第 i 个数据元素赋给 e
   if (!LocateElem(La, e, equal( )) )
      ListInsert(La, ++La_len, e);
      // La中不存在和 e 相同的数据元素，则插入之
   }
} // union
```

试改变结构，以**有序表**表示集合。

若线性表中的数据元素相互之间可以**比较**，并且数据元素在线性表中**依值非递减或非递增有序**排列，即

$a_i \geqslant a_{i-1}$ 或 $a_i \leqslant a_{i-1}(i = 2,3,\cdots, n)$，则称该线性表为**有序表**(Ordered List)。

例如：

（2，3，3，5，6，6，6，8，12）

对集合 B 而言，

值相同的数据元素必定相邻；

对集合 A 而言，

数据元素依值从小至大的顺序插入。

因此，**数据结构改变了，**

解决问题的策略也相应要改变。

```
void purge(List &La, List Lb) {
  InitList(LA);    La_len = ListLength(La);
  Lb_len =ListLength(Lb);    // 求线性表的长度
  for (i = 1;  i <= Lb_len;  i++) {
    GetElem(Lb, i, e); // 取Lb中第i个数据元素赋给 e
    if (ListEmpty(La) || !equal (en, e)) {
          ListInsert(La, ++La_len, e);
          en = e;
    } // La中不存在和 e 相同的数据元素，则插入之
  }
} // purge
```



例 2-3

已知线性表LA和LB的数据元素按值非递减有序排列。现要求将LA和LB并归为一个新的线性表LC,且LC的数据元素仍按值非递减有序排列。

```
void MergeList(List La, List Lb, List &Lc) {
  // 本算法将非递减的有序表 La 和 Lb 归并为 Lc
      InitList(Lc);  // 构造空的线性表 Lc
      i = j = 1;    k = 0; // i、j 指向两个待比较元素
      La_len = ListLength(La);
      Lb_len = ListLength(Lb);
      while ((i <= La_len) && (j <= Lb_len))
           {  // La 和 Lb 均不空 }
      while (i<=La_len)  // 若 La 不空
      while (j<=Lb_len)  // 若 Lb 不空
} // merge_list
```



```
 // La 和 Lb 均非空，i = j = 1, k = 0
GetElem(La, i, ai);
GetElem(Lb, j, bj);
if (ai <= bj) {  // 将 ai 插入到 Lc 中
   ListInsert(Lc, ++k, ai);  ++i; }
else {   // 将 bj 插入到 Lc 中
   ListInsert(Lc, ++k, bj);  ++j; }
```

```
while (i <= La_len) { // 当La不空时
    GetElem(La, i++, ai);
    ListInsert(Lc, ++k, ai);
}  // 插入 La 表中剩余元素

while (j <= Lb_len) { // 当Lb不空时
    GetElem(Lb, j++, bj);
    ListInsert(Lc, ++k, bj);
}  // 插入 Lb 表中剩余元素
```

2.2 线性表类型的实现——顺序映象

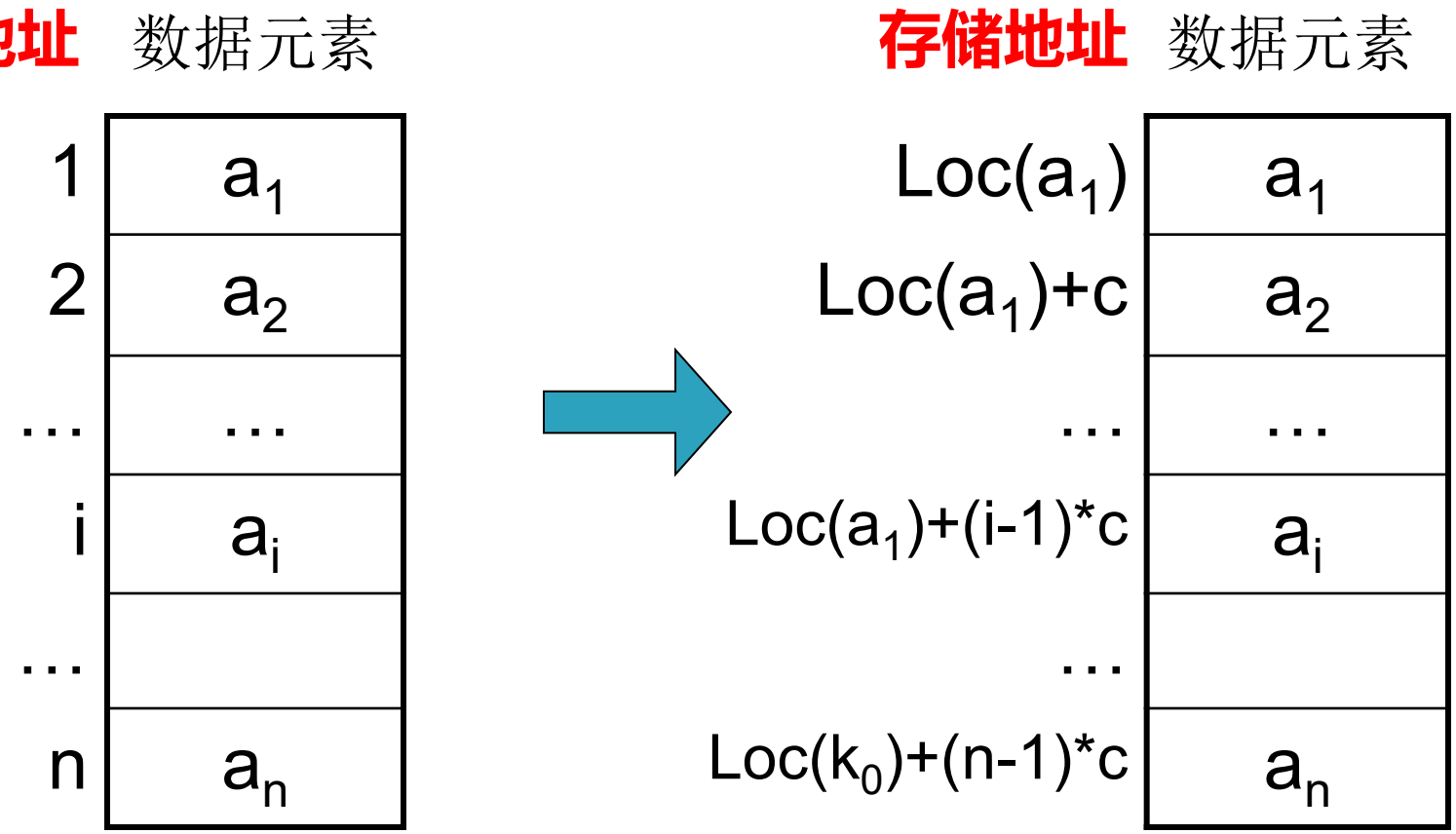
顺序映象

—— 以 x 的存储位置和 y 的存储位置之间某种关系表示逻辑关系<x,y>。

最简单的一种顺序映象方法是：

令 y 的存储位置和 x 的存储位置相邻。

线性表的顺序存储结构示意图



$$Locate(a_{i+1}) = Locate(a_i) + sizeof(ElemType)$$

$$Locate(a_i) = Locate(a_1) + sizeof(ElemType)$$

*(i−1)

基地址

顺序映像的 C 语言描述

```
#define  LIST_INIT_SIZE     80
                // 线性表存储空间的初始分配量
#define  LISTINCREMENT    10
                // 线性表存储空间的分配增量
typedef  struct {
     ElemType *elem;     // 存储空间基址
      int       length;   // 当前长度
      int       listsize;  // 当前分配的存储容量
                                // (以sizeof(ElemType)为单位)
} SqList;  // 俗称 顺序表
```

线性表的基本操作在顺序表中的实现

InitList(&L)　// 结构初始化

LocateElem(L, e, compare())　// 查找

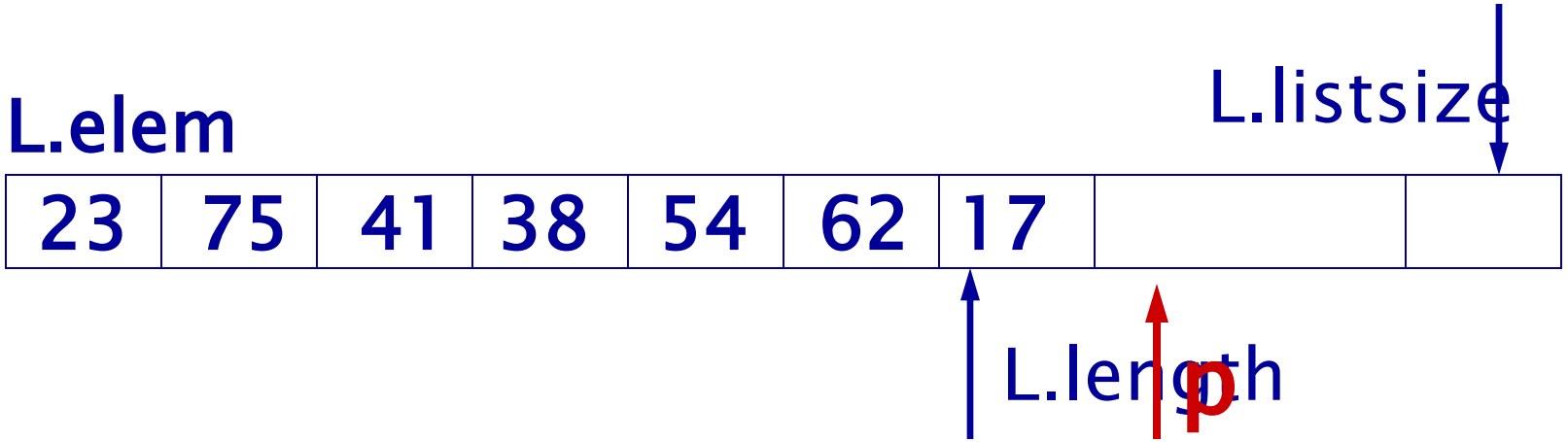
ListInsert(&L, i, e)　// 插入元素

ListDelete(&L, i)　// 删除元素

```
Status InitList_Sq( SqList& L ) {
  // 构造一个空的线性表
  L.elem = (ElemType*) malloc (LIST_
                  INIT_SIZE*sizeof (ElemType));
  if (!L.elem) exit(OVERFLOW);
  L.length = 0; // 已分配容量不等于元素个数
  L.listsize = LIST_INIT_SIZE
  return OK;
} // InitList_Sq
```

算法时间复杂度：O(1)

例如：顺序表



i 8

e =38 50

可见，基本操作是：将顺序表中的元素逐个和给定值 e 相比较。

```
int LocateElem_Sq(SqList L, ElemType e,
  Status (*compare)(ElemType, ElemType)) {
  // 在顺序表中查询第一个满足判定条件的数据元素，
 // 若存在，则返回它的位序，否则返回 0
  i = 1;          // i 的初值为第 1 元素的位序
  p = L.elem;       // p 的初值为第 1 元素的存储位置
  while (i <= L.length &&
                    !(*compare)(*p++, e))  ++i;
  if (i <= L.length)  return i;
  else  return 0;
} // LocateElem_Sq
```

算法的时间复杂度为：

O(ListLength(L))

线性表操作

ListInsert(&L, i, e)的实现：

首先分析:

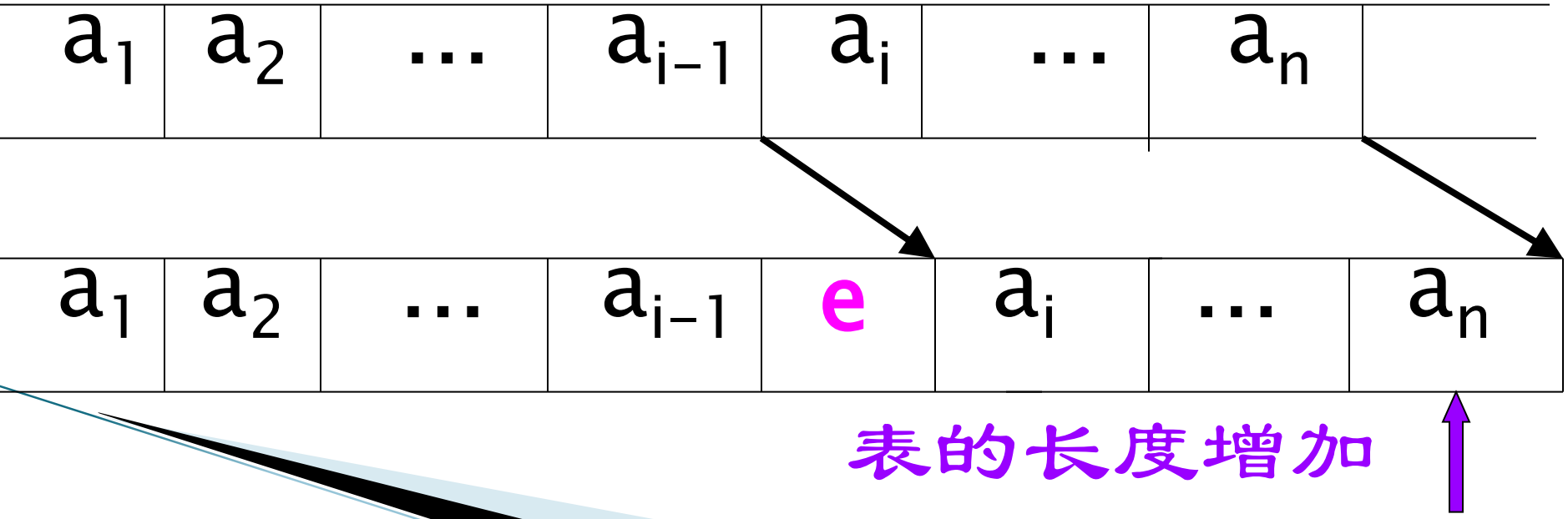
插入元素时，

线性表的**逻辑结构发生什么变化**?

$(a_1, \ldots, a_{i-1}, a_i, \ldots, a_n)$ 改变为

$(a_1, \ldots, a_{i-1}, e, a_i, \ldots, a_n)$

$<a_{i-1}, a_i> \longrightarrow <a_{i-1}, e>, <e, a_i>$



```
Status ListInsert_Sq(SqList &L, int i,
ElemType e) {
  // 在顺序表L的第 i 个元素之前插入新的元素e,
  // i 的合法范围为 1≤i≤L.length+1

q = &(L.elem[i-1]);                    // q 指示插入位置
for (p = &(L.elem[L.length-1]); p >= q; --p)
     *(p+1) = *p;  // 插入位置及之后的元素右移
*q = e;          // 插入e
++L.length;   // 表长增1
return OK;
```

算法时间复杂度为：

O(ListLength(L))

```
} // ListInsert_Sq
```

```
if (i < 1 || i > L.length+1) return ERROR;
                                        // 插入位置不合法
if (L.length >= L.listsize) {
                                // 当前存储空间已满，增加分配
   newbase = (ElemType *)realloc(L.elem,
      (L.listsize+LISTINCREMENT)*sizeof(ElemType));
   if (!newbase) exit(OVERFLOW);
                                // 存储分配失败
   L.elem = newbase;                    // 新基址
   L.listsize += LISTINCREMENT; // 增加存储容量
}
```

考虑移动元素的平均情况：

假设在第 i 个元素之前插入的概率为p_i ，则在长度为*n* 的线性表中**插入一个元素所需移动元素次数的期望值**为：

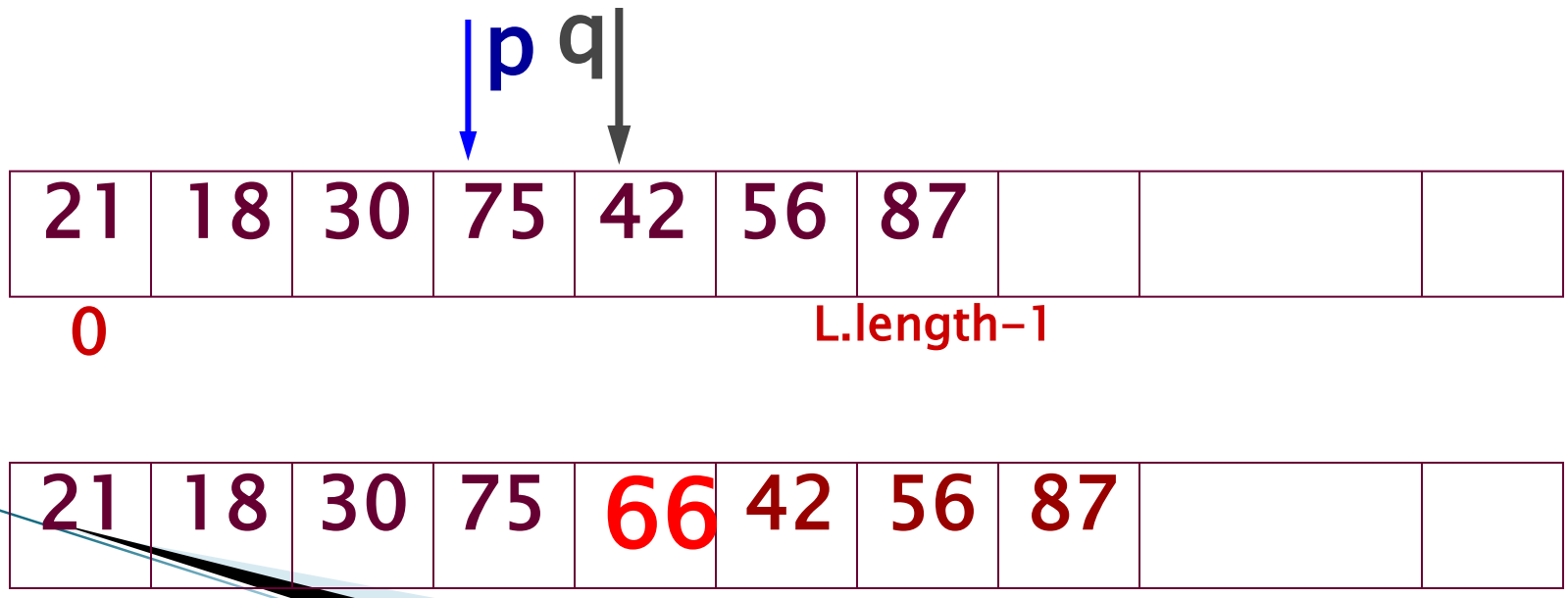
$$E_{is} = \sum_{i=1}^{n+1} p_i (n - i + 1)$$

若**假定**在线性表中任何一个位置上进行**插入的概率**都是**相等**的，则**移动元素的期望值**为：

$$E_{is} = \frac{1}{n+1} \sum_{i=1}^{n+1} (n - i + 1) = \frac{n}{2}$$

例如：ListInsert_Sq(L, 5, 66)

```
q = &(L.elem[i-1]);      // q 指示插入位置
for (p = &(L.elem[L.length-1]); p >= q;  --p)
    *(p+1) = *p;
```



线性表操作

ListDelete(&L, i, &e)的实现

首先分析：

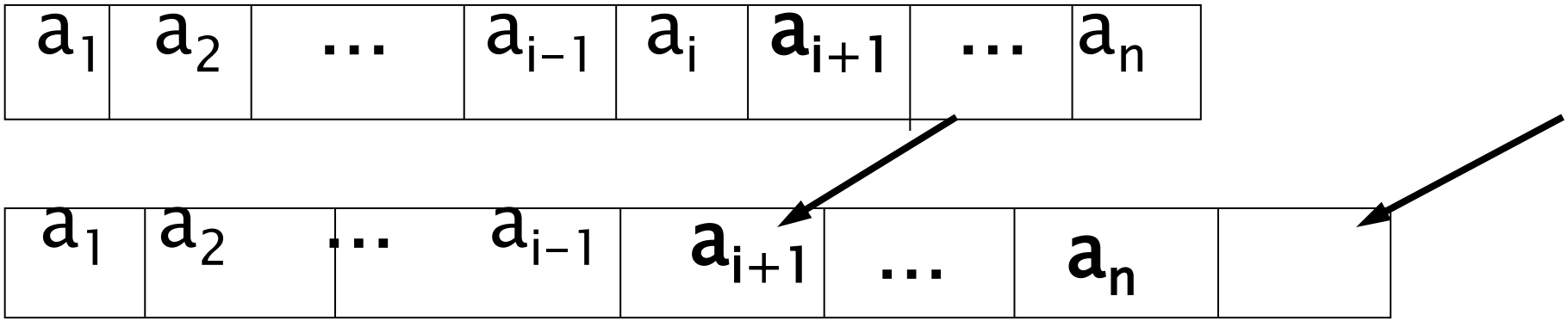
删除元素时，

线性表的逻辑结构发生什么变化?

$(a_1, \ldots, a_{i-1}, a_i, a_{i+1}, \ldots, a_n)$ 改变为

$(a_1, \ldots, a_{i-1}, a_{i+1}, \ldots, a_n)$

$<a_{i-1}, a_i>, <a_i, a_{i+1}>$ ⟹ $<a_{i-1}, a_{i+1}>$



```
Status ListDelete_Sq
                    (SqList &L, int i, ElemType &e) {
  if ((i < 1) || (i > L.length))  return ERROR;
                                        // 删除位置不合法
  p = &(L.elem[i-1]);       // p 为被删除元素的位置
  e = *p;                            // 被删除元素的值赋给 e
  q = L.elem+L.length-1;     // 表尾元素的位置

  for (++p; p <= q; ++p)  *(p-1) = *p;
                     // 被删除元素之后的元素左移
  --L.length;        // 表长减1
  return OK;
} // ListDelete_Sq
```

算法时间复杂度为：

O(ListLength(L))

考虑移动元素的平均情况：

假设删除第 i 个元素的概率为q_i ，则在长度为 *n* 的线性表中删除一个元素所需**移动元素次数的期望值**为：

$$E_{dl} = \sum_{i=1}^{n} q_i (n-i)$$

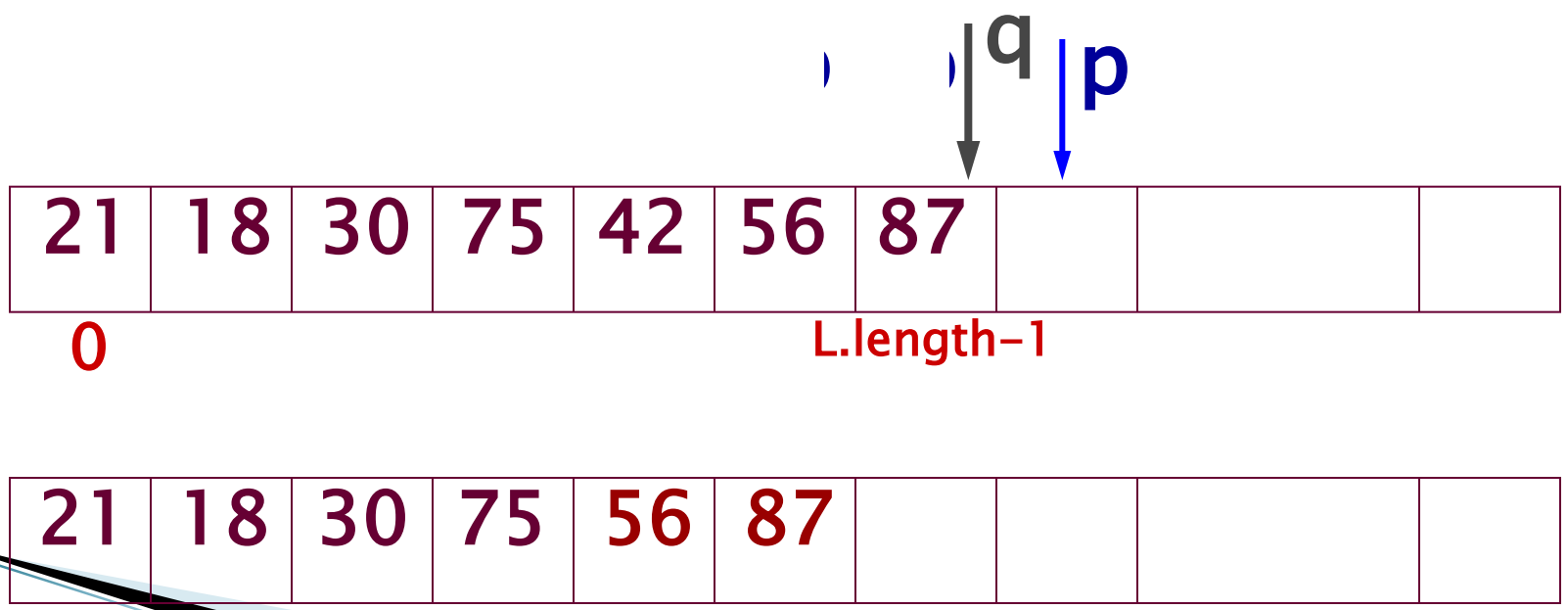
若假定在线性表中任何一个位置上进行删除的概率都是相等的，则**移动元素的期望值**为：

$$E_{dl} = \frac{1}{n} \sum_{i=1}^{n} (n-i) = \frac{n-1}{2}$$



例如：ListDelete_Sq(L, 5, e)

```
p = &(L.elem[i-1]);
q = L.elem+L.length-1;
for (++p; p <= q; ++p)  *(p-1) = *p;
```



顺序表的优缺点

- **顺序表的优点：**

(1) 无需为表示表中元素之间的逻辑关系而增加额外的存储空间；

(2) 随机存取：可以快速地存取表中任一位置的元素。

- **顺序表的缺点：**

(1) 插入和删除操作需要移动大量元素；

(2) 表的容量难以确定，表的容量难以扩充；

(3) 造成存储空间的碎片。

2.3 线性表类型的实现---链式映象

一、单链表

二、结点和单链表的 C 语言描述

三、线性表的操作在单链表中的实现

四、一个带头结点的单链表类型

五、其它形式的链表

六、静态链表

一、单链表

用一组**地址任意**的存储单元**存放**线性表中的数据元素。

以**元素(数据元素的映象)**

+ **指针(指示后继元素存储位置)**

= **结点**

(表示数据元素 或 **数据元素的映象**)

以“**结点的序列**”表示线性表

—— 称作**链表**

线性表的链式存储结构及实现

单链表

顺序表 → 静态存储分配 → 事先确定容量

链　表 → 动态存储分配 → 运行时分配空间

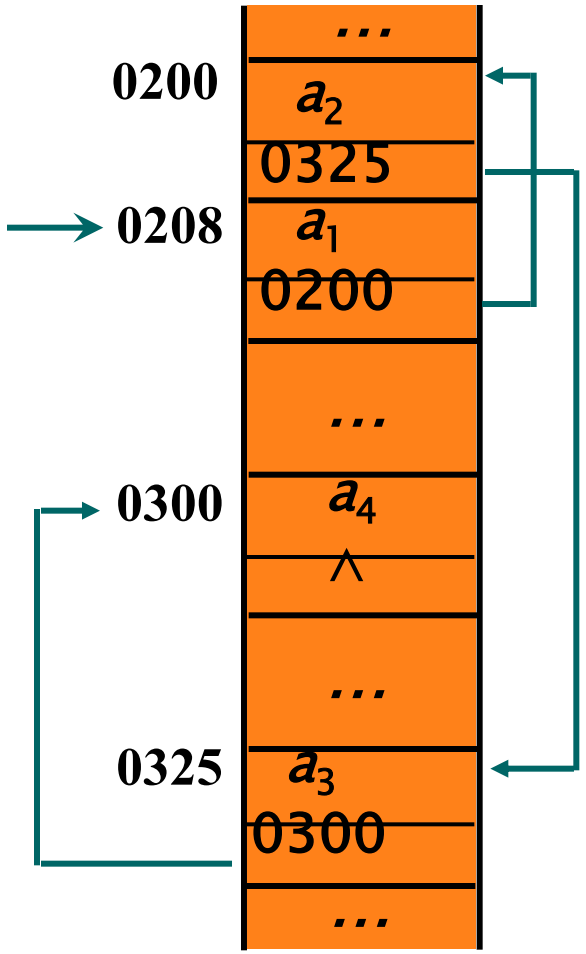
单链表： 线性表的链式存储结构。

存储思想： 用一组**任意**的存储单元存放线性表的元素。

- 连续
- 不连续
- 零散分布

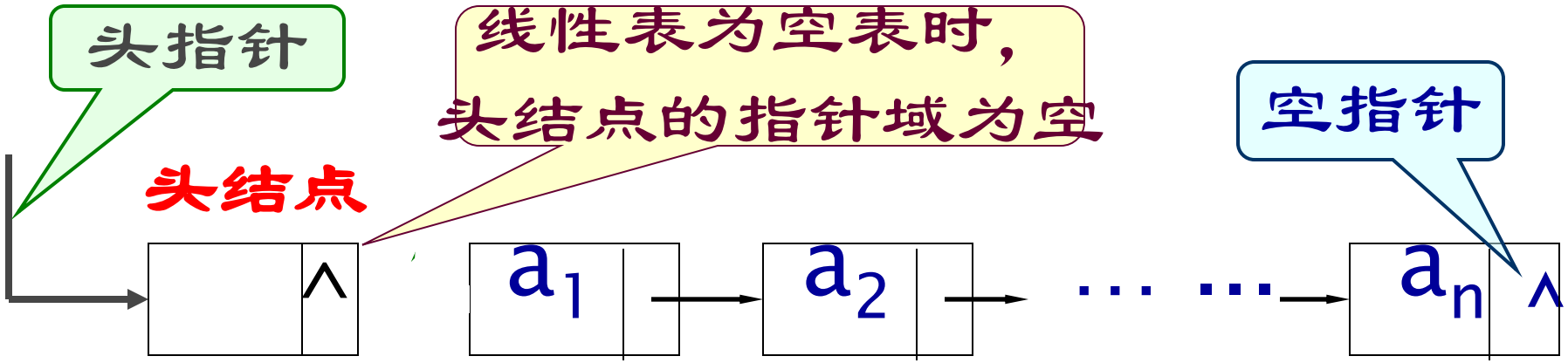
单链表

例：(a_1,a_2,a_3,a_4)的存储示意图



存储特点：

1.逻辑次序和物理次序
　不一定相同。
2.元素之间的逻辑关系
　用指针表示。



以线性表中第一个数据元素 a_1 的存储地址作为线性表的地址，称作线性表的**头指针**。

有时为了操作方便，在第一个结点之前虚加一个“头结点”，以**指向头结点的指针**为链表的头指针。

- 头结点的结构和链表中其它结点相同，只是它的数据域中不存放任何信息，但它的指针域存放的是第一个数据元素的存储地址，即线性表的基地址。此时称指向头结点的指针为头指针。通常称这类单链表为“带头结点的单链表”。
- 值得注意的是，若线性表为空，在不带头结点的情况下，头指针为空(NULL)，但在带头结点的情况下，链表的头指针不为空，而是其头结点中指针域的指针为空。

二、结点和单链表的 C 语言描述

```
Typedef struct  LNode {
    ElemType      data;   // 数据域
    struct Lnode   *next;   // 指针域
} LNode, *LinkList;
```

LinkList L;   // L 为单链表的头指针



结点和单链表的实现

结点是通过动态分配和释放来的实现，即需要时分配，不需要时释放。实现时是分别使用C语言提供的标准函数：malloc()，realloc()，sizeof()，free()。

动态分配　　p=(LNode*)malloc(sizeof(LNode));

函数malloc分配了一个类型为LNode的结点变量的空间，并将其首地址放入指针变量p中。

动态释放　free(p);

系统回收由指针变量p所指向的内存区。P必须是最近一次调用malloc函数时的返回值。

最常用的基本操作及其示意图

(1) 结点的赋值

```
LNode  *p;
p=(LNode*)malloc(sizeof(LNode));
p->data=20;  p->next=NULL ;
```

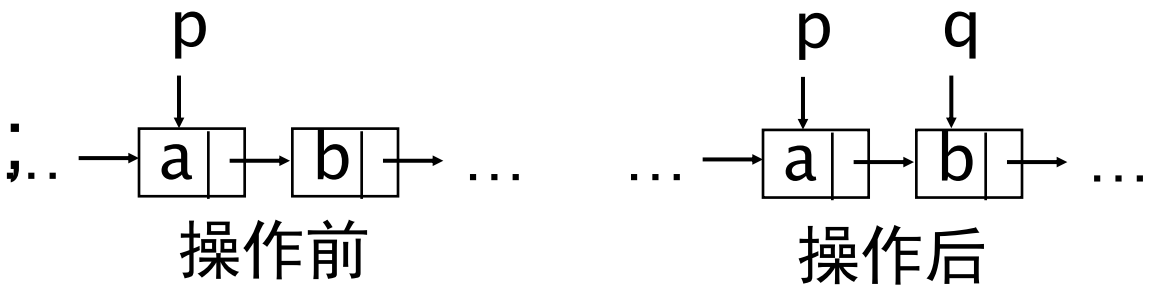


(2) 常见的指针操作

① q=p；



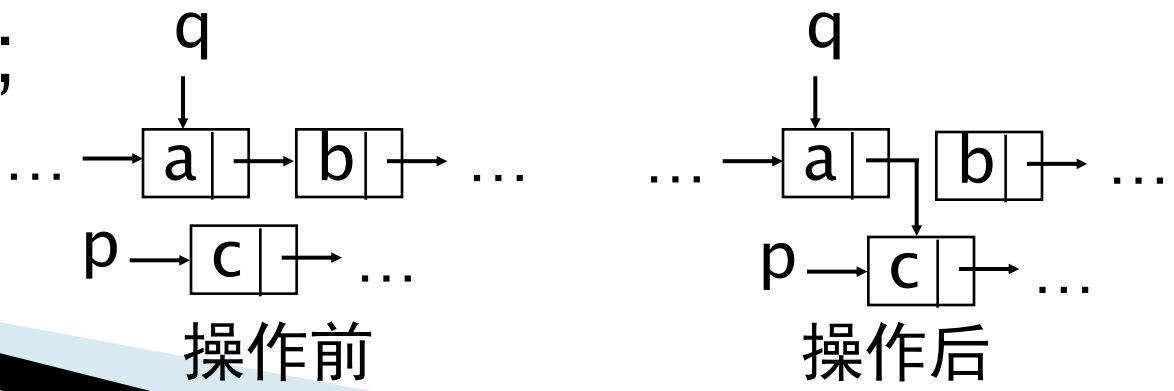
② q=p->next；



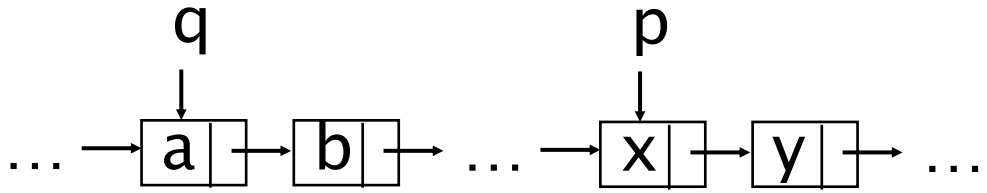
③ p=p->next；



④ q->next=p；

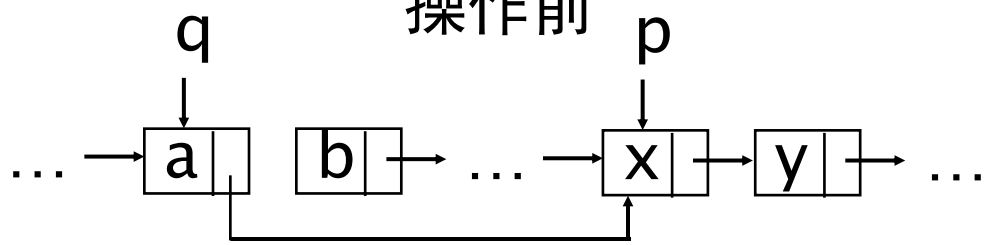


(a)



操作前

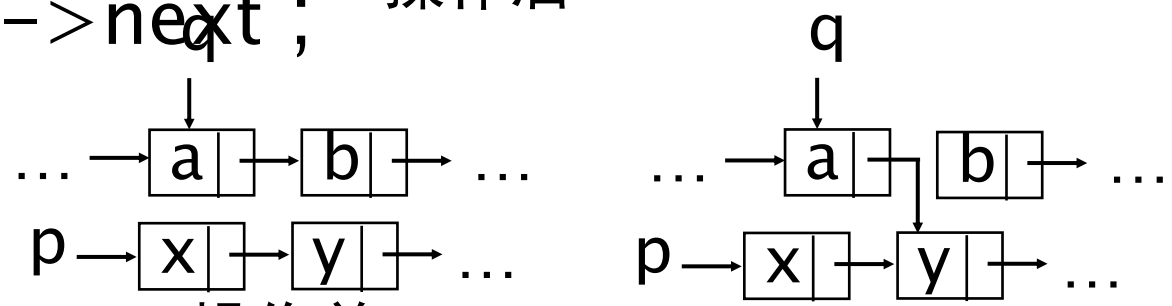
(b)



操作后

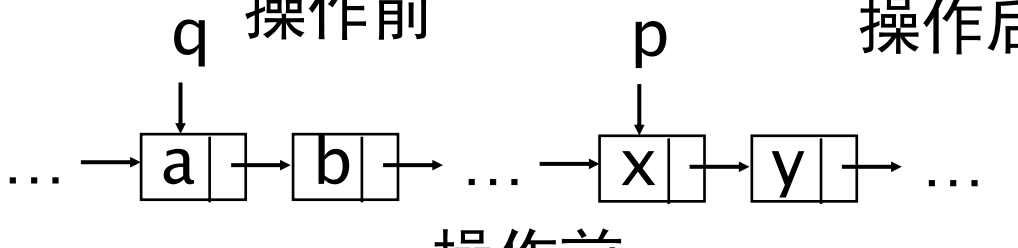
⑤ q->next=p->next ;

(a)

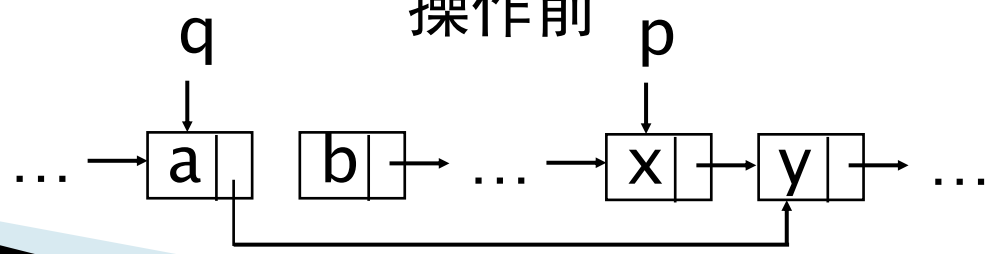


操作前　　操作后

(b)



操作前



操作后

三、单链表操作的实现

GetElem(L, i, e)　　// 取第i个数据元素

ListInsert(&L, i, e)　　// 插入数据元素

ListDelete(&L, i, e)　　// 删除数据元素

ClearList(&L)　　// 重置线性表为空表

CreateList(&L, n)

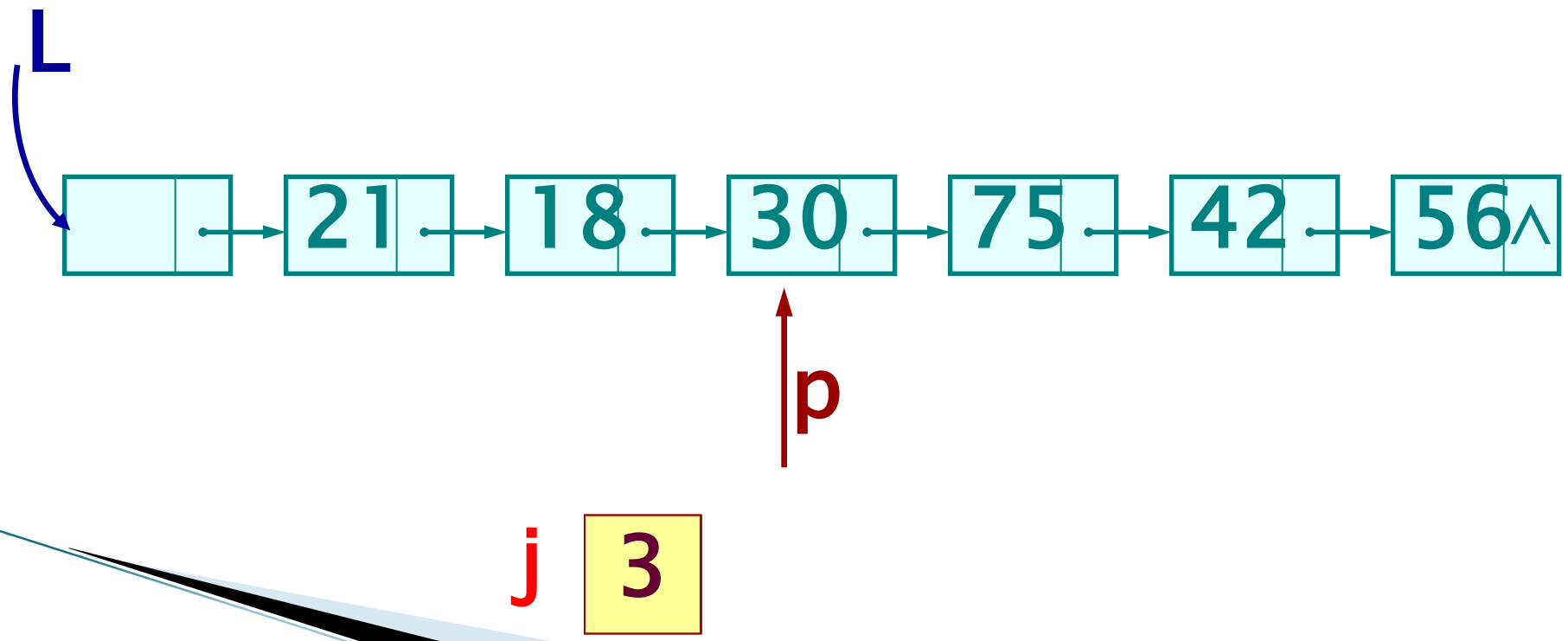
// 生成含 *n* 个数据元素的链表



线性表的操作

GetElem(L, i, &e)

在单链表中的实现:



单链表是一种顺序存取的结构，为找第 i 个数据元素，必须先找到第 i-1 个数据元素。

因此，查找第 i 个数据元素的基本操作为：移动指针，比较 j 和 i 。

令指针 p 始终指向线性表中第 j 个数据元素。

```
Status GetElem_L(LinkList L, int i, ElemType &e) {
  // L是带头结点的链表的头指针，以 e 返回第 i 个元素

 p = L->next;   j = 1;  // p指向第一个结点，j为计数器

 while (p && j<i)  { p = p->next;  ++j;  }
        // 顺指针向后查找，直到 p 指向第 i 个元素
     // 或 p 为空

 if ( !p || j>i )
     return ERROR;       //  第 i 个元素不存在
 e = p->data;                 //  取得第 i 个元素
 return OK;
} // GetElem_L
```

算法时间复杂度为：

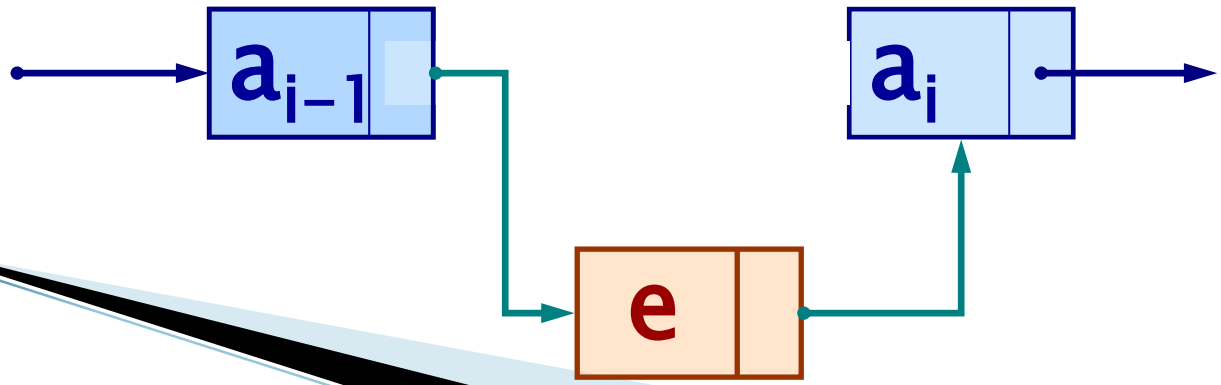
O(ListLength(L))

线性表的操作 **ListInsert(&L, i, e)**

在单链表中的实现：

有序对 $<a_{i-1}, a_i>$

改变为 $<a_{i-1}, e>$ 和$<e, a_i>$



可见，在链表中插入结点只需要修改指针。但同时，若要在第 i 个结点之前插入元素，修改的是第 i–1 个结点的指针。

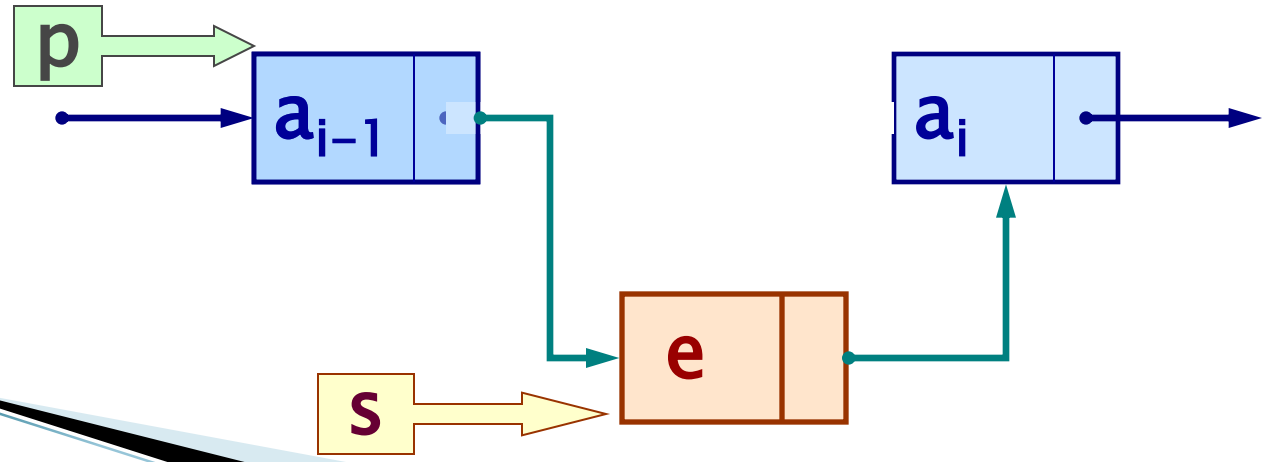
因此，在单链表中第 i 个结点之前进行插入的基本操作为:

找到线性表中第i-1个结点，然后修改其指向后继的指针。

```
Status ListInsert_L(LinkList L, int i, ElemType e) {
  // L 为带头结点的单链表的头指针，本算法
  // 在链表中第i 个结点之前插入新的元素 e
  p = L;    j = 0;
  while (p && j < i-1)
      { p = p->next;  ++j; }   // 寻找第 i-1 个结点
  if (!p || j > i-1)
       return ERROR;      // i 大于表长或者小于1
     ......
} // LinstInsert_L
```

算法的时间复杂度为: O(ListLength(L))

```
s = (LinkList) malloc ( sizeof (LNode));
                                    // 生成新结点
s->data = e;
s->next = p->next;        p->next = s; // 插入
return OK;
```



线性表的操作ListDelete (&L, i, &e)

在链表中的实现:

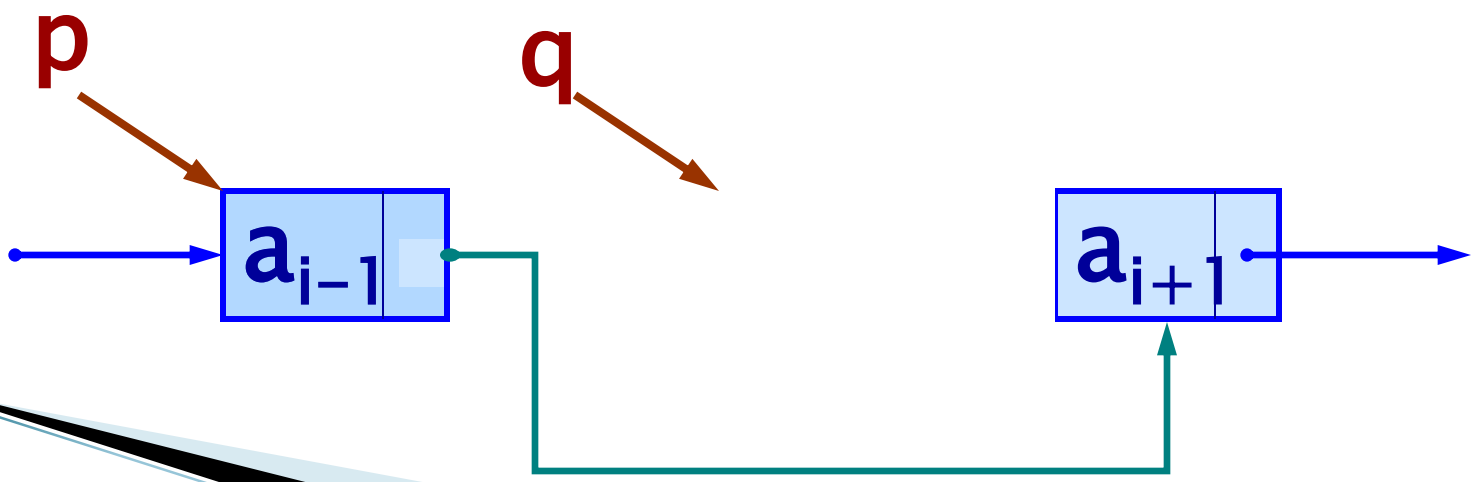
有序对$<a_{i-1}, a_i>$ 和 $<a_i, a_{i+1}>$

改变为 $<a_{i-1}, a_{i+1}>$



在单链表中删除第 i 个结点的基本操作为:找到线性表中第i-1个结点，修改其指向后继的指针。

```
q = p->next;   p->next = q->next;
e = q->data;   free(q);   // 释放前先取出数据
```



```
Status ListDelete_L(LinkList L, int i, ElemType &e) {
  // 删除以 L 为头指针(带头结点)的单链表中第 i 个结点

  p = L;    j = 0;
  while (p->next && j < i-1) {  p = p->next;   ++j; }
                          // 寻找第 i 个结点，并令 p 指向其前趋
  if  (!(p->next) || j > i-1)
      return ERROR;  // 删除位置不合理

  q = p->next;   p->next = q->next;  // 删除并释放结点
  e = q->data;   free(q);
  return OK;

} // ListDelete_L
```

算法的时间复杂度为：O(ListLength(L))

ClearList(&L) 在链表中的实现:

```
void ClearList(&L) {
    // 将单链表重新置为一个空表
    while (L->next) {
        p=L->next;    L->next=p->next;
        free(p);
    }
} // ClearList
```

算法时间复杂度： O(ListLength(L))

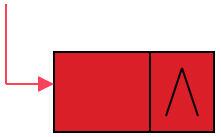
如何从线性表得到单链表?

链表是一个动态的结构，它不需要予分配空间，因此**生成链表的过程**是一个结点“**逐个插入**” 的过程。

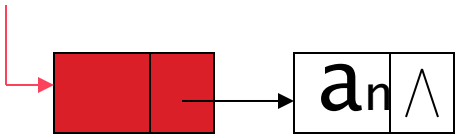
例如：逆位序输入 n 个数据元素的值，建立带头结点的单链表。

操作步骤：

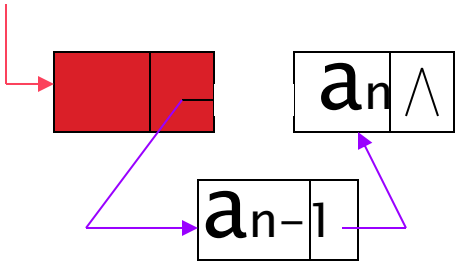
一、建立一个“空表”；



二、输入数据元素a_n，建立结点并插入；



三、输入数据元素a_{n-1}，建立结点并插入；



四、依次类推，直至输入a_1为止。

```
void CreateList_L(LinkList &L, int n) {
    // 逆序输入 n 个数据元素，建立带头结点的单链表

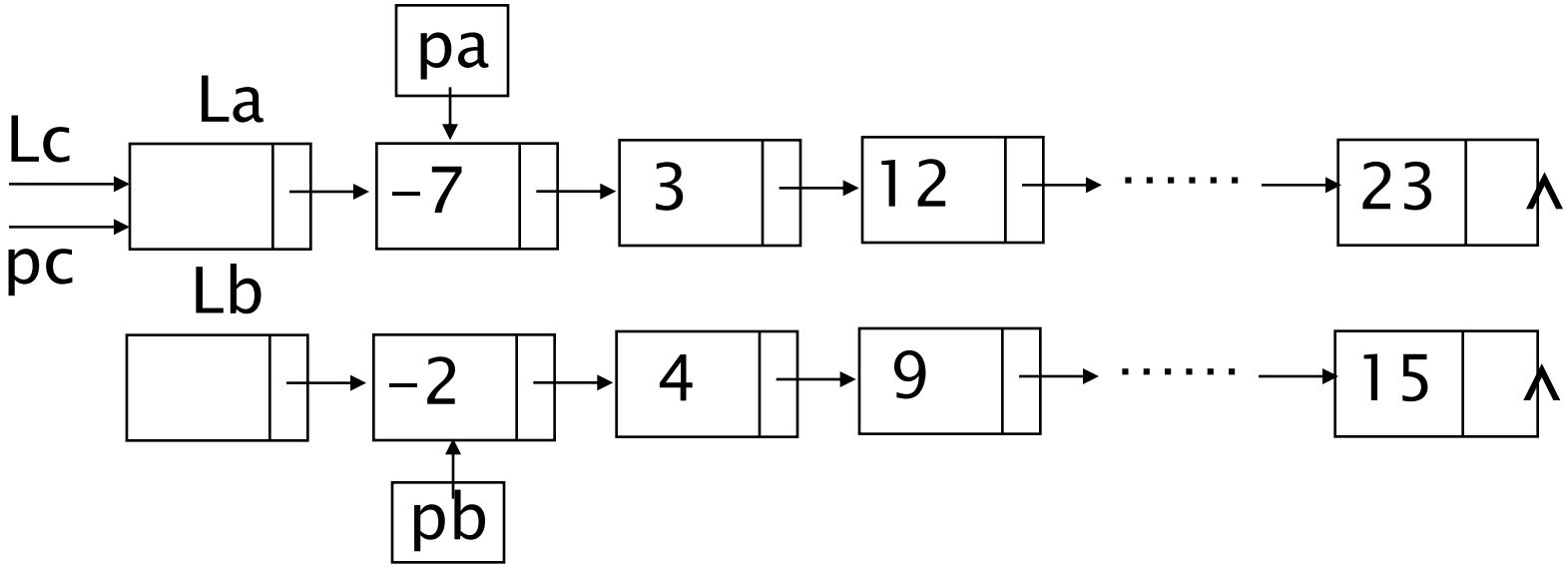
    L = (LinkList) malloc (sizeof (LNode));
    L->next = NULL;     // 先建立一个带头结点的单链表

    for (i = n; i > 0; --i) { // 逐次头插，链表顺序与读入相反
        p = (LinkList) malloc (sizeof (LNode));
        scanf(&p->data);     // 输入元素值
        p->next = L->next; L->next = p;  // 插入
    }
} // CreateList_L
```

算法的时间复杂度为: O(Listlength(L))

单链表的合并

设有两个有序的单链表，它们的头指针分别是La 、 Lb，将它们合并为以Lc为头指针的有序链表。合并前的示意图如图2-4所示。



两个有序的单链表La，Lb的初始状态

合并了值为-7，-2的结点后示意图如图2-5所示。

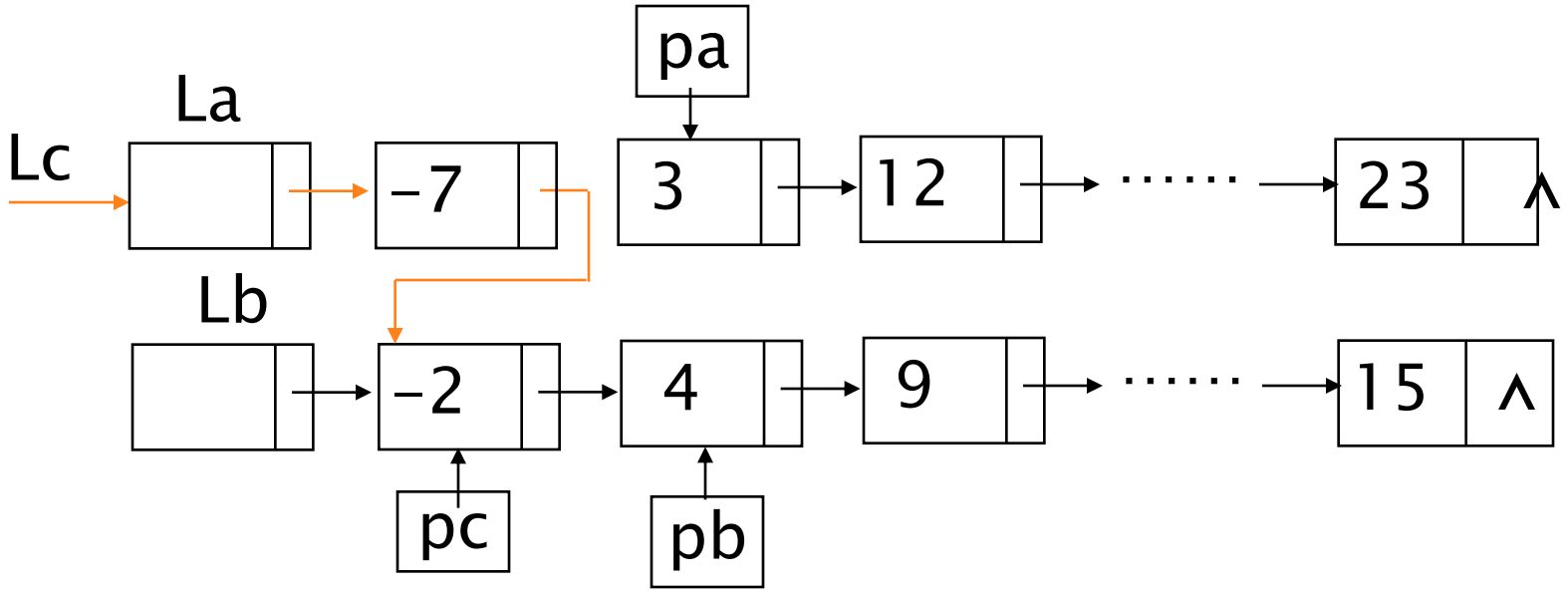


图2-5　合并了值为-7，-2的结点后的状态

算法说明

算法中pa，pb分别是待考察的两个链表的当前结点，pc是合并过程中合并的链表的最后一个结点。

算法描述

```
LNode  *Merge_LinkList(LNode *La,  LNode *Lb)
    /*  合并以La, Lb为头结点的两个有序单链表  */
  {   LNode *Lc,  *pa ,  *pb ,  *pc, *ptr ;
      Lc=La ;  pc=La  ;   pa=La->next ;  pb=Lb->next  ;
      while (pa!=NULL        && pb!=NULL)
        {  if  (pa->data<pb->data)
             {   pc->next=pa ;  pc=pa ;   pa=pa->next  ;  }
           /*  将pa所指的结点合并，pa指向下一个结点  */
           if  (pa->data>pb->data)
             {   pc->next=pb ;  pc=pb ;   pb=pb->next  ;  }
           /*  将pa所指的结点合并，pa指向下一个结点  */
```

```
            if (pa->data==pb->data)
               {  pc->next=pa ; pc=pa ;  pa=pa->next ;
                   ptr=pb ; pb=pb->next ; free(ptr) ;  }
            /* 将pa所指的结点合并，pb所指结点删除 */
          }
       if (pa!=NULL) pc->next=pa ;
      else  pc->next=pb ;     /*将剩余的结点链上*/
      free(Lb) ;
      return(Lc) ;
   }
```

算法分析

若La，Lb两个链表的长度分别是m，n，则链表合并的时间复杂度为O(m+n)。

用上述定义的单链表实现线性表的操作时，存在的问题：

1．单链表的表长是一个隐含的值；

2．在单链表的最后一个元素之后插入元素时，需遍历整个链表；

3．在链表中，元素的“位序”概念淡化，结点的“位置”概念加强。

改进链表的设置：

1．增加“表长”、“表尾指针”和“当前位置的指针”三个数据域；

2．将基本操作中的“位序 i ”改变为“指针 p ”。

四、一个带头结点的线性链表类型

```
typedef struct LNode {   // 结点类型
    ElemType        data;
    struct LNode    *next;
} *Link, *Position;

Status MakeNode( Link &p, ElemType e );
    // 分配由 p 指向的值为e的结点，并返回OK，
    // 若分配失败，则返回 ERROR

void    FreeNode( Link &p );
    // 释放 p 所指结点
```

```
typedef struct {   // 链表类型
   Link  head, tail;      // 分别指向头结点和
                                  // 最后一个结点的指针
   int   len;                 // 指示链表长度
   Link  current;     // 指向当前被访问的结点
                                  //的指针，初始位置指向头结点
} LinkList;
```

链表的基本操作:

{结构初始化和销毁结构}

```
Status InitList( LinkList &L );
  // 构造一个空的线性链表 L，其头指针、
  // 尾指针和当前指针均指向头结点，
  // 表长为零。
```

O(1)

```
Status DestroyList( LinkList &L );
  // 销毁线性链表 L，L不再存在。
```

O(n)

{引用型操作}

```
Status ListEmpty ( LinkList L );  //判表空
```
O(1)

```
int  ListLength( LinkList L );     // 求表长
```
O(1)

```
Status Prior( LinkList L );
    // 改变当前指针指向其前驱
```
O(n)

```
Status Next ( LinkList L );
    // 改变当前指针指向其后继
```
O(1)

```
ElemType GetCurElem ( LinkList L );
    // 返回当前指针所指数据元素
```
O(1)

Status LocatePos(LinkList L, int i);

// 改变当前指针指向第i个结点

O(n)

Status LocateElem (LinkList L, ElemType e,
Status (*compare)(ElemType, ElemType));

O(n)

// 若存在与e 满足函数compare( )判定关系的元
// 素，则移动当前指针指向第1个满足条件的
// 元素的前驱，并返回OK; 否则返回ERROR

Status ListTraverse(LinkList L, **Status**(*visit)());

// 依次对L的每个元素调用函数visit()

O(n)

{加工型操作}

```
Status ClearList ( LinkList &L );
    // 重置 L 为空表
```

O(n)

```
Status SetCurElem(LinkList &L, ElemType e );
    // 更新当前指针所指数据元素
```

O(1)

```
Status Append ( LinkList &L, Link s );
    // 在表尾结点之后链接一串结点
```

O(s)

```
Status InsAfter ( LinkList &L, Elemtype e );
    // 将元素 e 插入在当前指针之后
```

O(1)

```
Status DelAfter ( LinkList &L, ElemType& e );
    // 删除当前指针之后的结点
```

O(1)

```
Status InsAfter( LinkList& L, ElemType e ) {
  // 若当前指针在链表中，则将数据元素e插入在线性链
  // 表L中当前指针所指结点之后，并返回OK;
   // 否则返回ERROR。

    if ( ! L.current )  return ERROR;
    if (! MakeNode( s, e) )  return ERROR;
    s->next = L.current->next;
    L.current->next = s;
    if (L.tail = L.current)  L.tail = s;
    L.current = s;     return OK;
} // InsAfter
```

```
Status DelAfter( LinkList& L, ElemType& e ){
  // 若当前指针及其后继在链表中，则删除线性链表L中当
  // 指针所指结点之后的结点，并返回OK；否则返回ERR
    if ( !(L.current && L.current->next ) )
          return ERROR;
    q = L.current->next;
    L.current->next = q->next;
    if (L.tail = q)  L.tail = L.current;
    e=q->data;  FreeNode(q);
    return OK;
} //DelAfter
```

利用上述定义的线性链表如何完成 线性表的其它操作 ？

例一

```
Status ListInsert_L(LinkList L, int i, ElemType e) {
  // 在带头结点的单链线性表 L 的第 i 个元素之前插入元素 e

  if (!LocatePos (L, i-1))  return ERROR;
                        // i 值不合法，第 i-1 个结点不存在
  if (InsAfter (L, e))  return OK;        // 完成插入
  else return ERROR;
} // ListInsert_L
```

例二

```
Status MergeList_L(LinkList &Lc, LinkList &La
                   LinkList &Lb ， int (*compare)
                   (ElemType,ElemType))) {
  // 归并有序表 La 和 Lb ,生成新的有序表 Lc,
  // 并在归并之后销毁La 和 Lb,
  // compare 为指定的元素大小判定函数

      ……

} // MergeList_L
```

```
if ( !InitList(Lc))  return ERROR;     // 存储空间分配失败
LocatePos (La, 0);  LocatePos (Lb, 0);   // 当前指针指向头结点
if ( DelAfter( La, e))  a = e;          // 取得 La 表中第一个元素 a
else a = MAXC;        // MAXC为常量最大值
if ( DelAfter( Lb, e))  b = e;          // 取得 Lb 表中第一个元素 b
else b = MAXC;        // a 和 b 为两表中当前比较元素

while (!( a=MAXC && b=MAXC)) {   // La 或 Lb 非空
            … …
}
DestroyList(La);  DestroyList(Lb);   // 销毁链表 La 和 Lb
return OK;
```

```
if ((*compare)(a, b) <=0) {  //  a≤b
    InsAfter(Lc, a);
    if ( DelAfter( La, e1) )    a = e1;
    else a = MAXC;
}
else {   //  a>b
    InsAfter(Lc, b);
    if ( DelAfter( Lb, e1) )    b = e1;
    else  b = MAXC;
}
```

五、其它形式的链表

1.双向链表

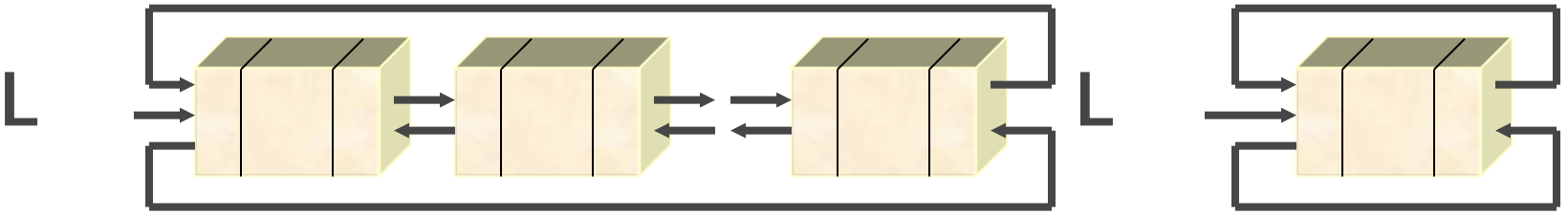
```
typedef struct  DuLNode {
    ElemType          data;    // 数据域
    struct DuLNode   *prior;
                                  // 指向前驱的指针域
    struct DuLNode   *next;
                                  // 指向后继的指针域
} DuLNode, *DuLinkList;
```

双向链表

- 双向链表是指在前驱和后继方向都能游历（遍历）的线性链表。
- 双向链表每个结点结构：



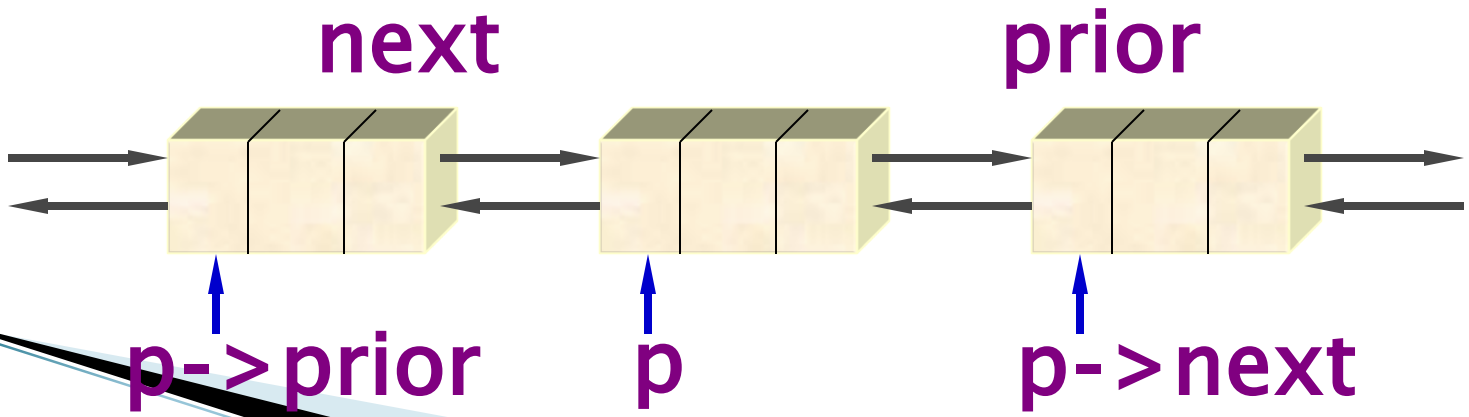
- 双向链表通常采用带表头结点的循环链表形式。



非空表　　空表

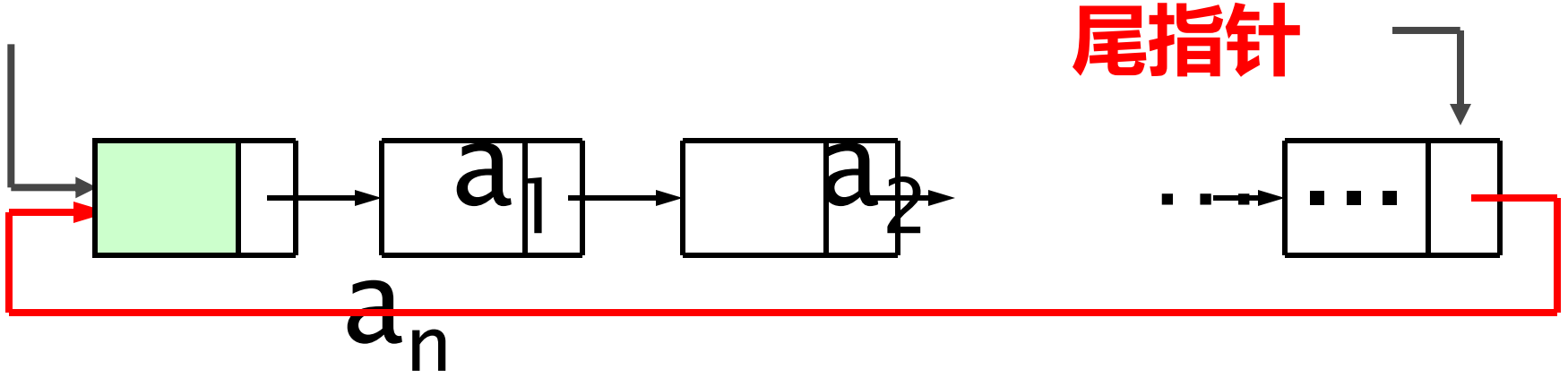
- **结点指向**

  p == p->prior >next== p-> next >prior



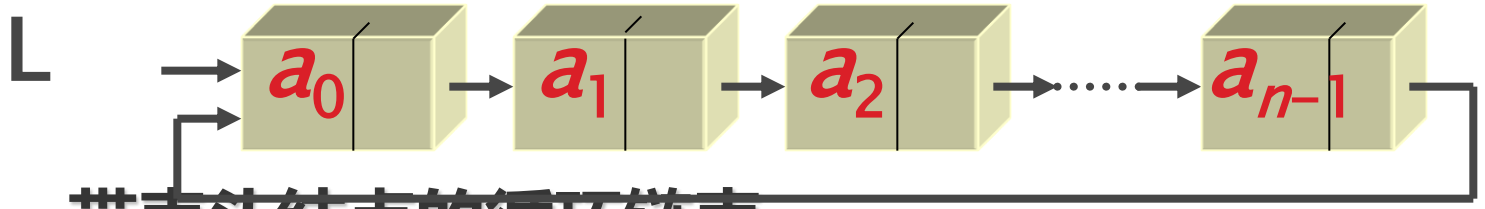
2. 循环链表（Circular Linked List）

最后一个结点的指针域的指针又指回第一个结点的链表

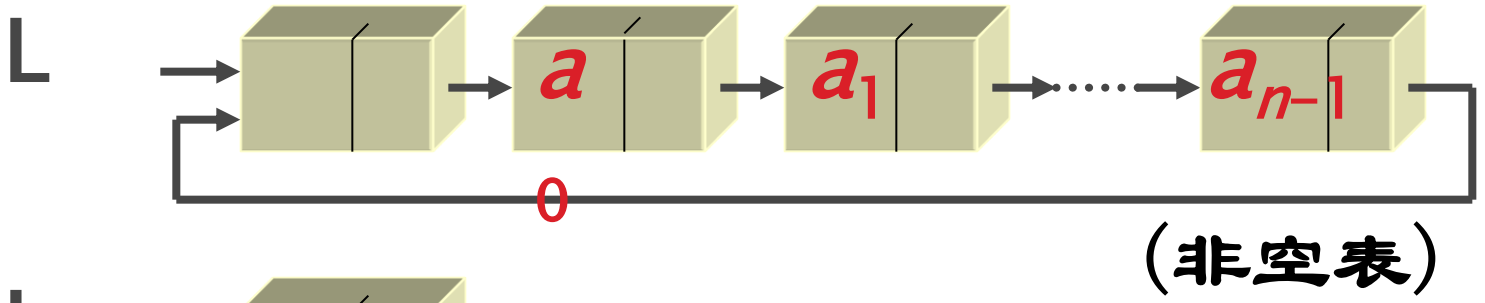


和单链表的差别仅在于，判别链表中最后一个结点的条件不再是“后继是否为空”，而是“后继是否为头结点”。

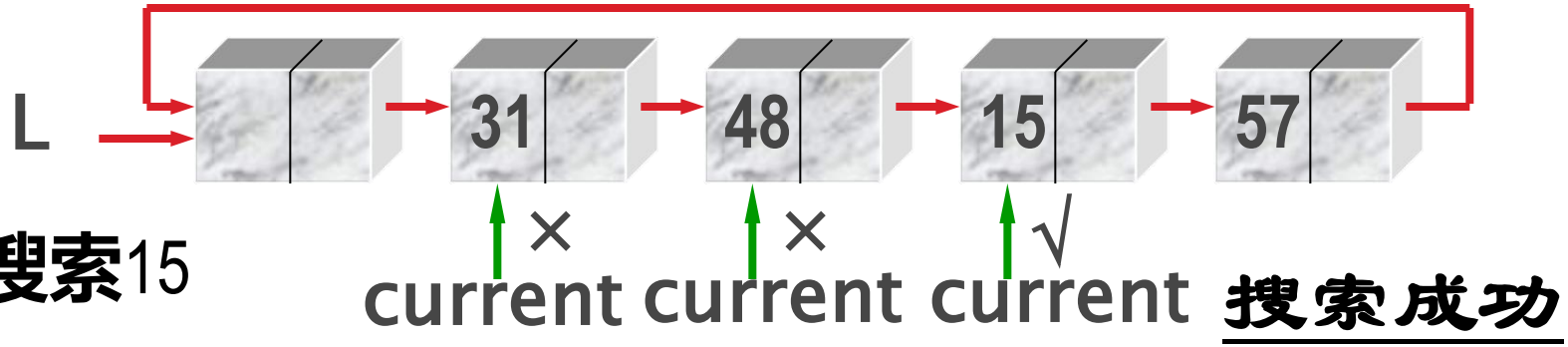
- 循环链表的示例

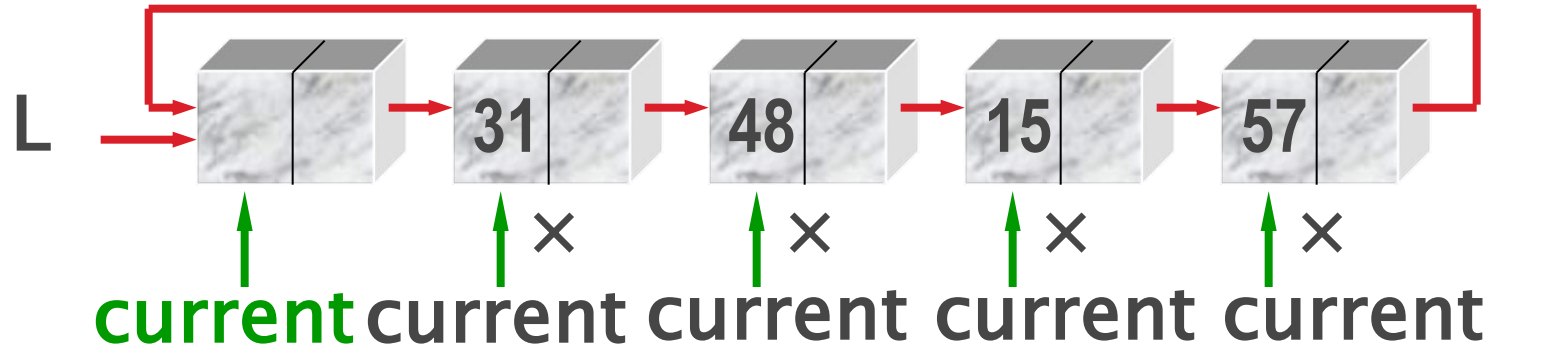


- 带表头结点的循环链表

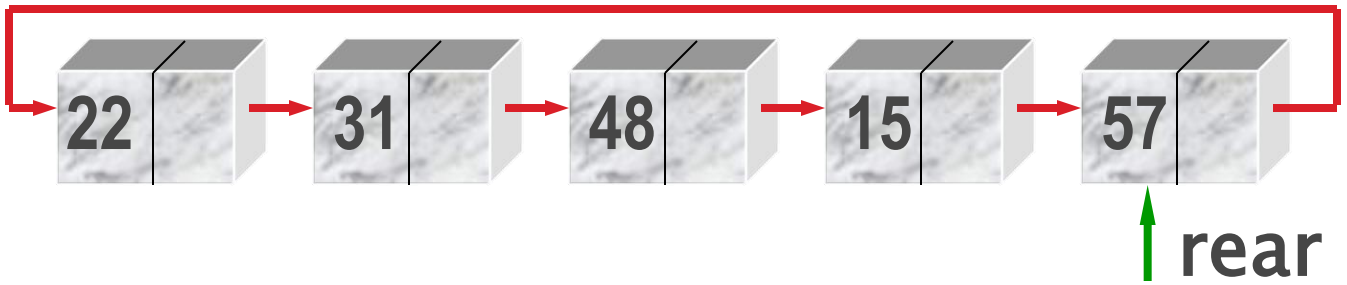


循环链表的搜索算法


L
31
48
15
57
×
×
√
current
current
current
搜索15
搜索成功


L
31
48
15
57
×
×
×
×
current
current
current
current
current
搜索25
搜索不成功

带尾指针的循环链表

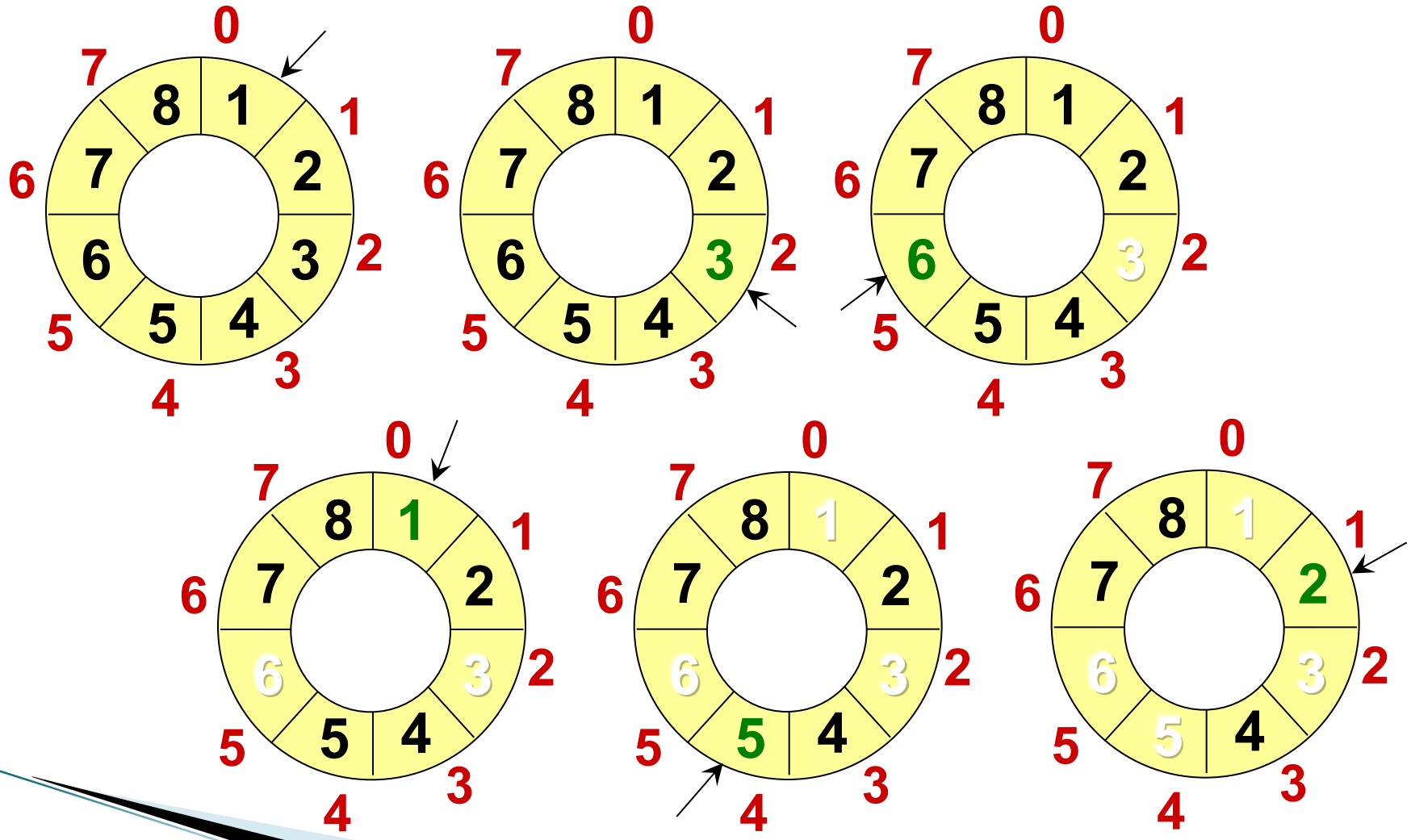


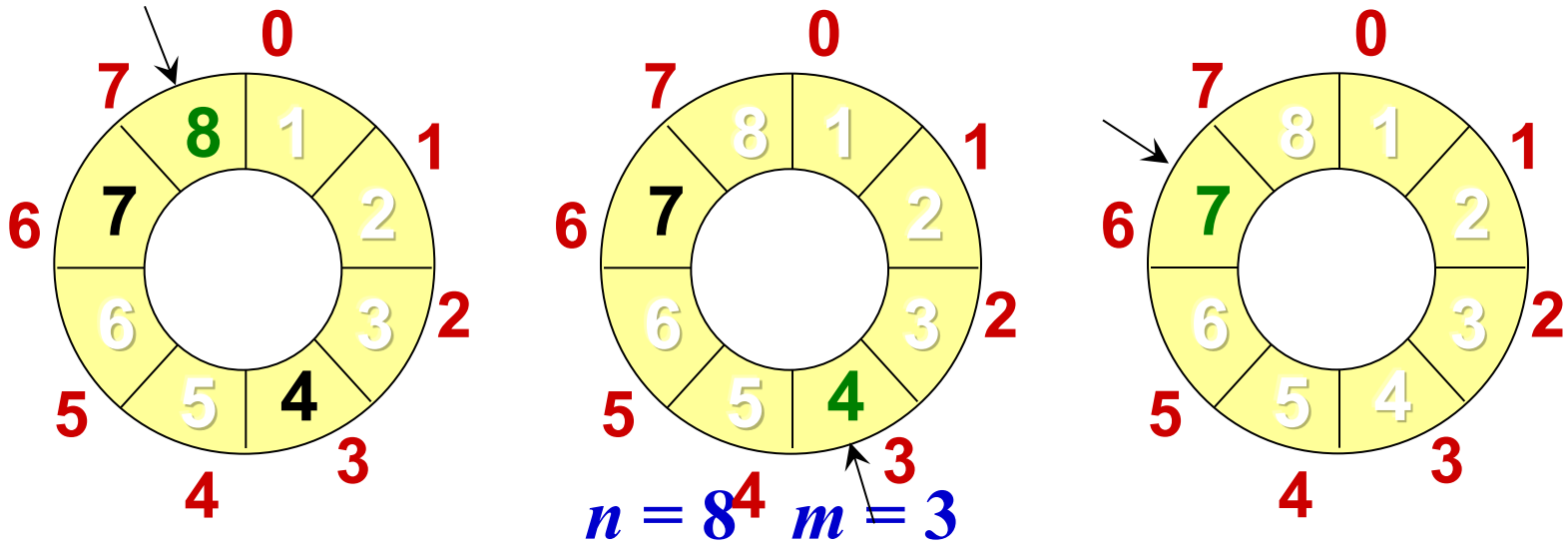
- 如果插入与删除仅在链表的两端发生，可采用带表尾指针的循环链表结构。
  - 在表尾插入，时间复杂性 O(1)
  - 在表尾删除，时间复杂性 O(n)
  - 在表头插入，相当于在表尾插入
  - 在表头删除，时间复杂性 O(1)

用循环链表求解约瑟夫问题

- **约瑟夫问题的提法**
  - *n* 个人围成一个圆圈，首先第 1 个人从 1 开始，一个人一个人顺时针报数，报到第 *m* 个人，令其出列。然后再从下一 个人开始，从 1 顺时针报数，报到第 *m* 个人，再令其出列，…，如此下去，直到圆圈中只剩一个人为止。此人即为优胜者。
  - 用不带表头结点的循环链表来组织。

例如 $n = 8$ $m = 3$


0 1 2 3 4 5 6 7
8 1 2 3 4 5 6 7
0 1 2 3 4 5 6 7
8 1 2 3 4 5 6 7
0 1 2 3 4 5 6 7
8 1 2 3 4 5 6 7
0 1 2 3 4 5 6 7
8 1 2 3 4 5 6 7
0 1 2 3 4 5 6 7
8 1 2 3 4 5 6 7

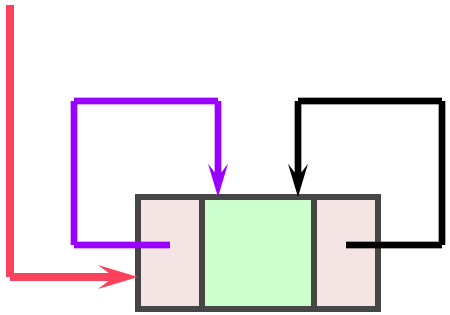

0
1
2
3
4
5
6
7
8
1
2
3
4
5
6
7
n = 8
m = 3

求解Josephus问题的算法

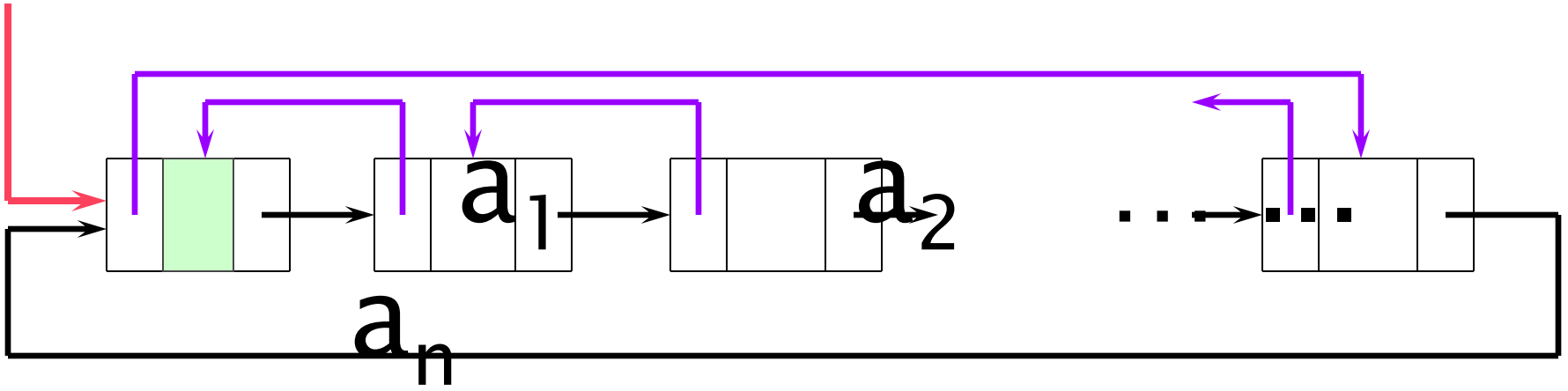
```
void Josephus(CircLinkList &Js, int n, int m) {
    CircLinkNode *p = Js.getHead(),
                                *pre = NULL;
    int i, j;
    for ( i = 0; i < n-1; i++ ) {   //执行n-1次
        for ( j = 1; j < m; j++)    //数m-1个人
            { pre = p;  p = p->next; }
        printf( "出列的人是" + p->data);
        pre->next = p->next;  delete p;          //删去
        p = pre->next; // 从出列者的后继重新报数
    }
};
```

双向循环链表

空表



非空表

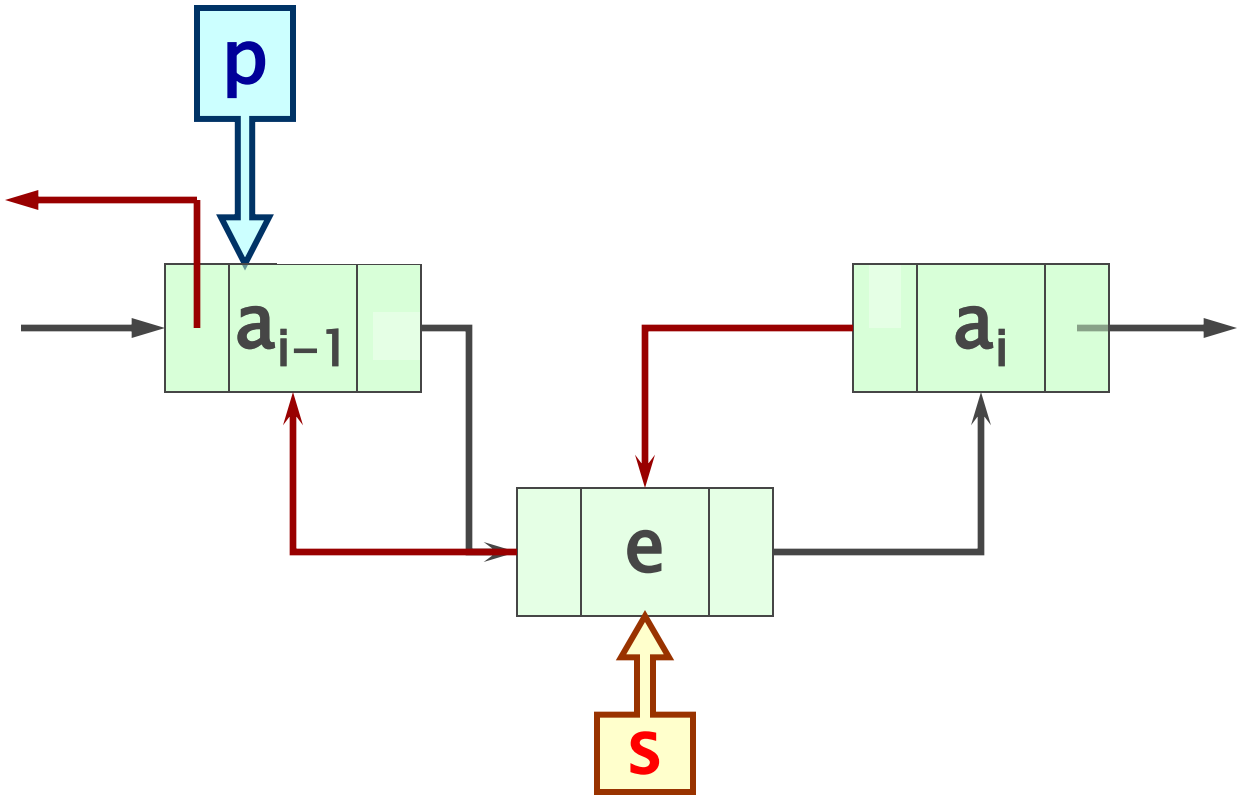


双向链表的操作特点：

“查询” 和单链表相同。

“插入” 和“删除”时需要同时修改两个方向上的指针。

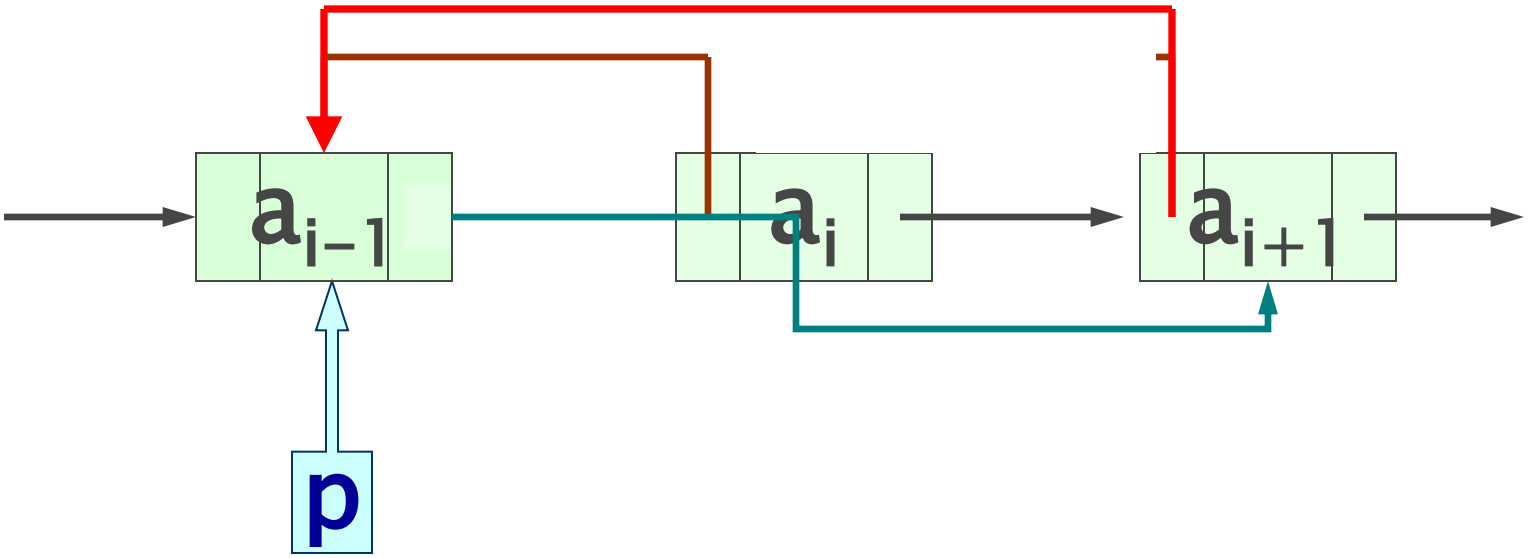
插入



s->next = p->next;　　p->next = s;

s->next->prior = s;　　s->prior = p;

删除




```
p->next = p->next->next;
```

```
p->next->prior = p;
```

六. 静态链表

起因：一些语言不支持“指针”类型，不能动态生成结点，故借用一维数组来描述线性链表，便于在不支持指针类型的高级程序设计语言中使用链表结构。

类型定义

```
#define   MAXSIZE    1000
typedef struct{
 ElemType data;
 int cur; //替代动态链表结点中的指针域
}component, SLinkList[MAXSIZE];
```

与动态链表使用上的差异

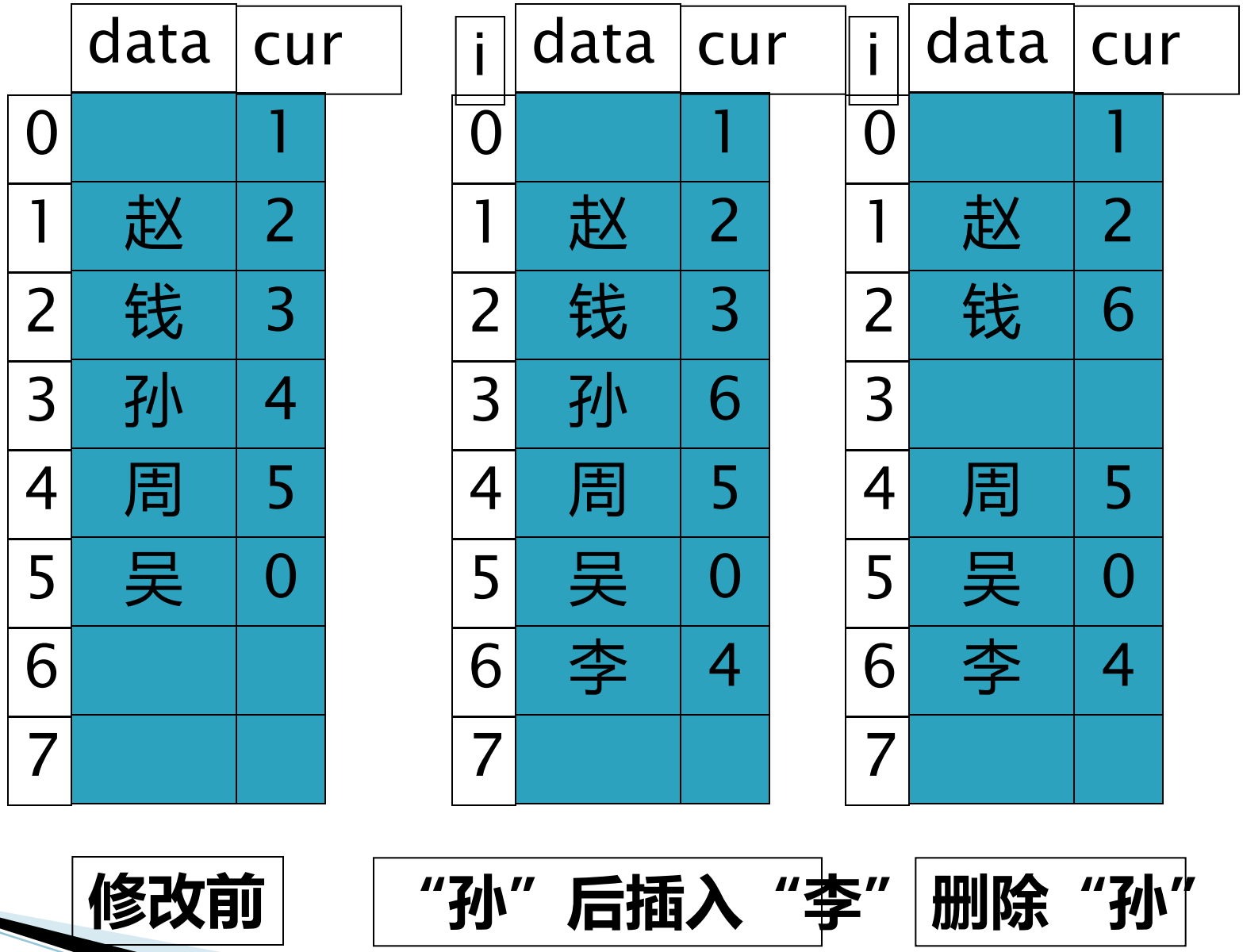
1)链表的结点是数组中的一个分量；

2)数组分量中的cur代替动态链表中的指针域,用来 指示直接后继或直接前驱结点在数组中的相对位置；

3) 数组的第0个分量可视为头结点；

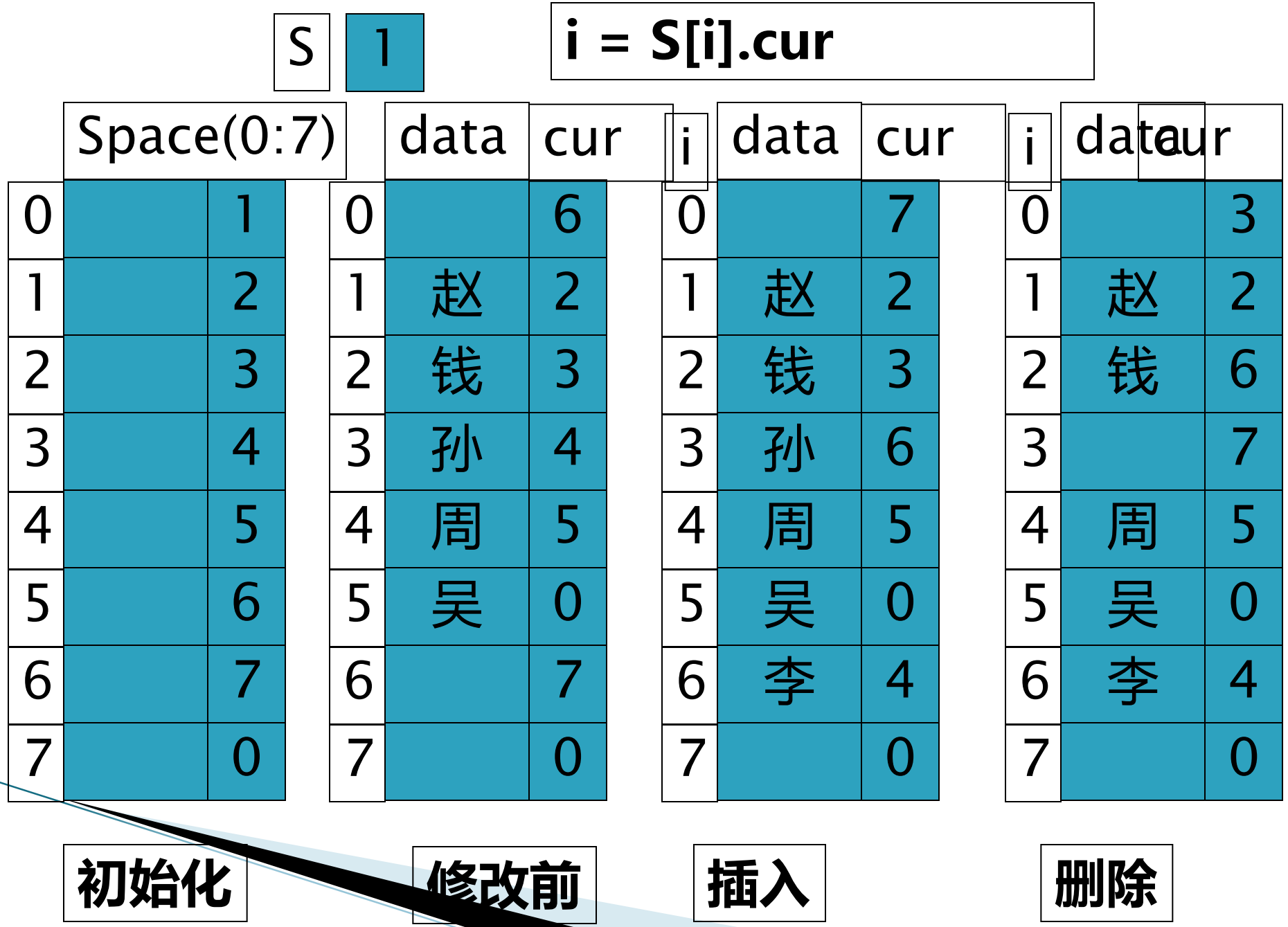
4)以0代表空指针NULL

5)以整型游标i代替动态指针p，以i = S[i].cur代替p = p->next取下一个元素的位置


data
cur
0
1
赵
2
钱
3
孙
4
周
5
吴
0
6
7
i
data
cur
0
1
赵
2
钱
3
孙
6
周
5
吴
0
李
4
7
i
data
cur
0
1
赵
2
钱
6
3
周
5
吴
0
6
李
4
7
修改前
“孙”后插入“李”
删除“孙”

2、动态分配与释放的模拟

思想：将未使用的分量用游标链成一个备用链；插入时，取备用链中的第一个结点作为插入的新结点；删除时将从链表中删除下来的结点链接到备用链上。


S
1
i = S[i].cur
Space(0:7)
data
cur
i
data
cur
i
data
cur
0 1 0 6 0 7 0 3
1 2 1 赵 2 1 赵 2 1 赵 2
2 3 2 钱 3 2 钱 3 2 钱 6
3 4 3 孙 4 3 孙 6 3 7
4 5 4 周 5 4 周 5 4 周 5
5 6 5 吴 0 5 吴 0 5 吴 0
6 7 6 7 6 李 4 6 李 4
7 0 7 0 7 0
初始化
修改前
插入
删除

初始化链表

space[0].cur为备用链的头指针，

cur域为0表示空指针

```
void InitSpace_SL(SlinkList &space){
   for( i=0 ; i < MAXSIZE-1; i++)
   space[i].cur=i+1;
     space[MAXSIZE-1] = 0;
```

模拟动态分配

```
int Malloc_SL(SlinkList &space){
  i = space[0].cur; // 取备用链的首个可用下标
if ( space[0].cur!=0 )    space[0].cur = space[i].cur;
   // 0表示为空
   return i;
}
```

模拟动态释放

```
void Free_SL(SlinkList &space, int k){
space[k].cur = space[0].cur;  space[0].cur = k;
}
```

2.4 一元多项式的表示

一元多项式

$$p_n(x) = p_0 + p_1 x + p_2 x^2 + \ldots + p_n x^n$$

在计算机中，可以用一个线性表来表示:

$P = (p_0, p_1, \ldots, p_n)$

但是对于形如

$S(x) = 1 + 3x^{10000} - 2x^{20000}$

的多项式，上述表示方法是否合适?

一般情况下的**一元稀疏多项式**可写成

$$P_n(x) = p_1x^{e1} + p_2x^{e2} + \cdots + p_mx^{em}$$

其中： p_i 是指数为e_i 的项的非零系数，

$$0 \le e_1 < e_2 < \cdots < e_m = n$$

可以下列线性表表示：

$$((p_1, e_1) , (p_2, e_2), \cdots, (p_m,e_m))$$

例如:

$$P_{999}(x) = 7x^3 - 2x^{12} - 8x^{999}$$

可用线性表

$$((7, 3), (-2, 12), (-8, 999))$$

表示

抽象数据类型一元多项式的定义

ADT Polynomial {

数据对象：

D={ a_i | a_i ∈TermSet, i=1,2,...,m, m≥0

TermSet 中的**每个元素包含一个表示系数的实数和表示指数的整数** }

数据关系：

R1={ <a_{i-1} ,a_i >|a_{i-1} ,a_i∈D, i=2,...,n

且a_{i-1}**中的指数值**＜a_i**中的指数值** }

基本操作：

CreatPolyn (&P, m)

操作结果：输入 m 项的系数和指数，建立一元多项式 P。

DestroyPolyn (&P)

初始条件：一元多项式 P 已存在。

操作结果：销毁一元多项式 P。

PrintPolyn (&P)

初始条件：一元多项式 P 已存在。

操作结果：打印输出一元多项式 P。

PolynLength(P)

初始条件：一元多项式 P 已存在。

操作结果：返回一元多项式 P 中的项数。

AddPolyn (&Pa, &Pb)

初始条件：一元多项式 Pa 和 Pb 已存在。

操作结果：完成多项式相加运算，即：

Pa = Pa + Pb，并销毁一元多项式 Pb

SubtractPolyn (&Pa, &Pb)

… …

} ADT Polynomial

一元多项式的实现：

```
typedef  OrderedLinkList   polynomial;
   // 用带表头结点的有序链表表示多项式
```

结点的数据元素类型定义为：

```
typedef struct {        // 项的表示
    float  coef;         // 系数
    int    expn;         // 指数
} term, ElemType;
```

```
Status CreatPolyn ( polynomail &P, int m ) {
 // 输入m项的系数和指数，建立表示一元多项式的有序链表P
    InitList (P); e.coef = 0.0;  e.expn = -1;
    SetCurElem (h, e);  // 设置头结点的数据元素
    for ( i=1; i<=m; ++i ) {  // 依次输入 m 个非零项
        scanf (e.coef, e.expn);
        if (!LocateElem ( P, e, (*cmp)()) )
            if ( !InsAfter ( P, e ) )  return ERROR;
    }
    return OK;
} // CreatPolyn
```

注意: 1.输入次序不限;

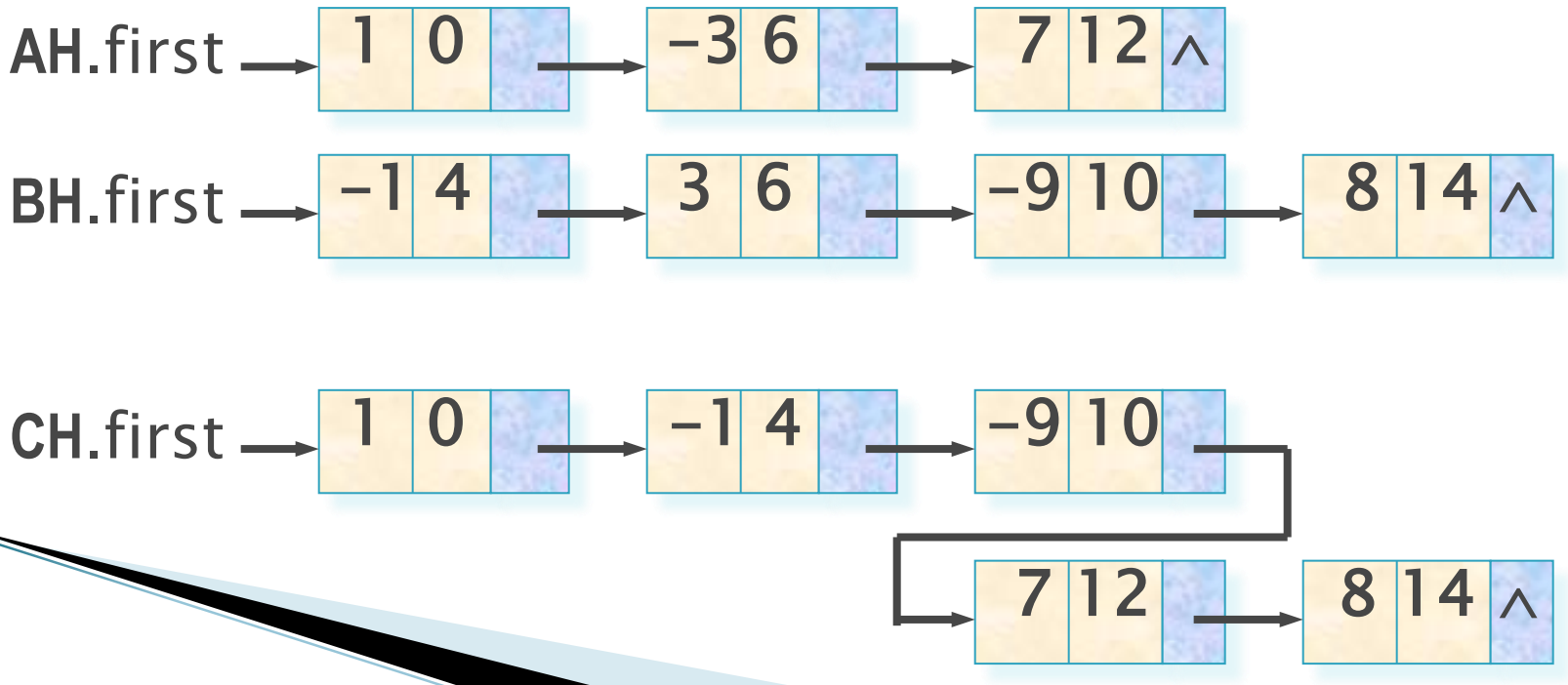
2.指数相同的项只能输入一次

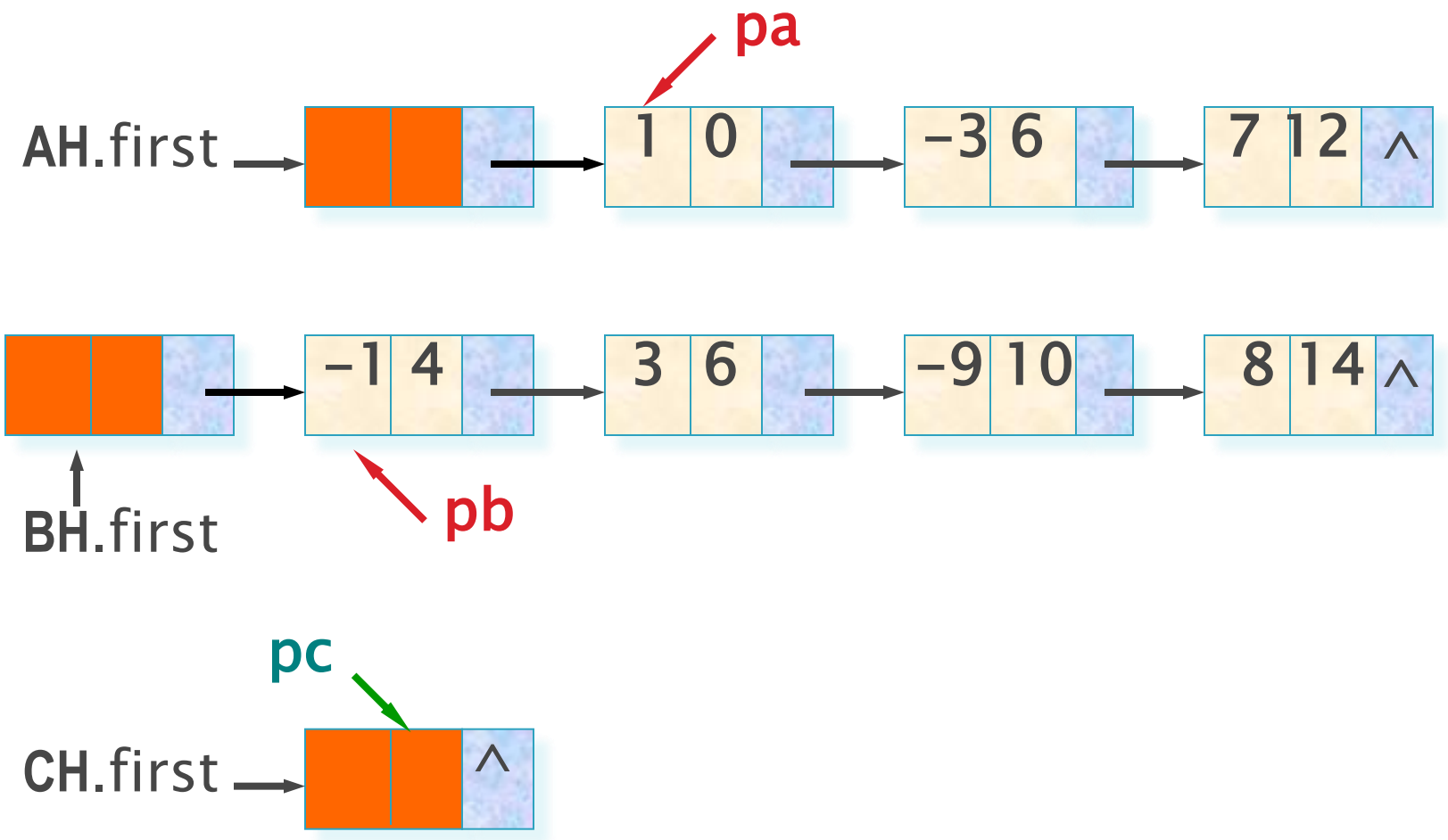
多项式链表的相加

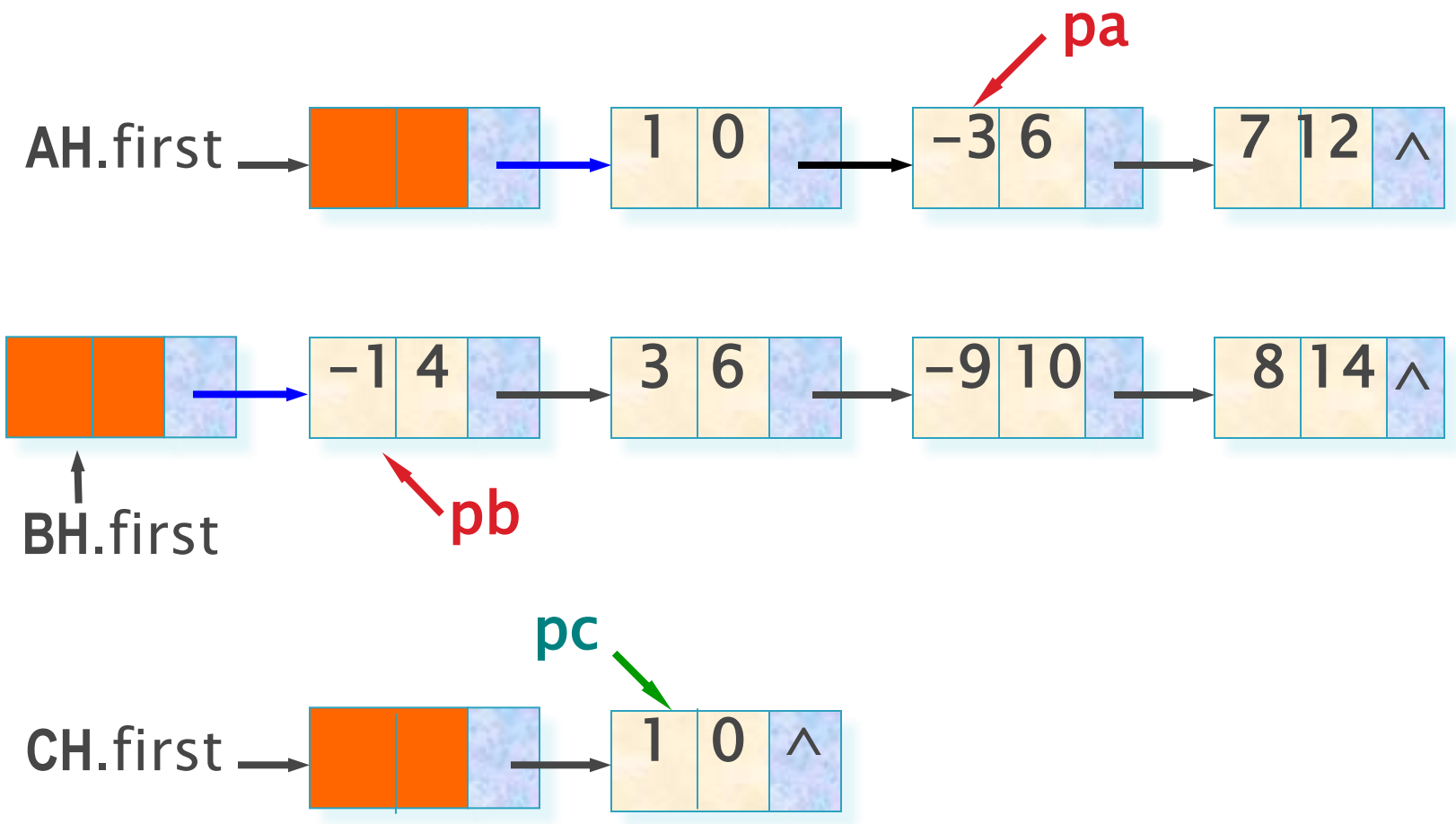
$AH = 1 - 3x^6 + 7x^{12}$

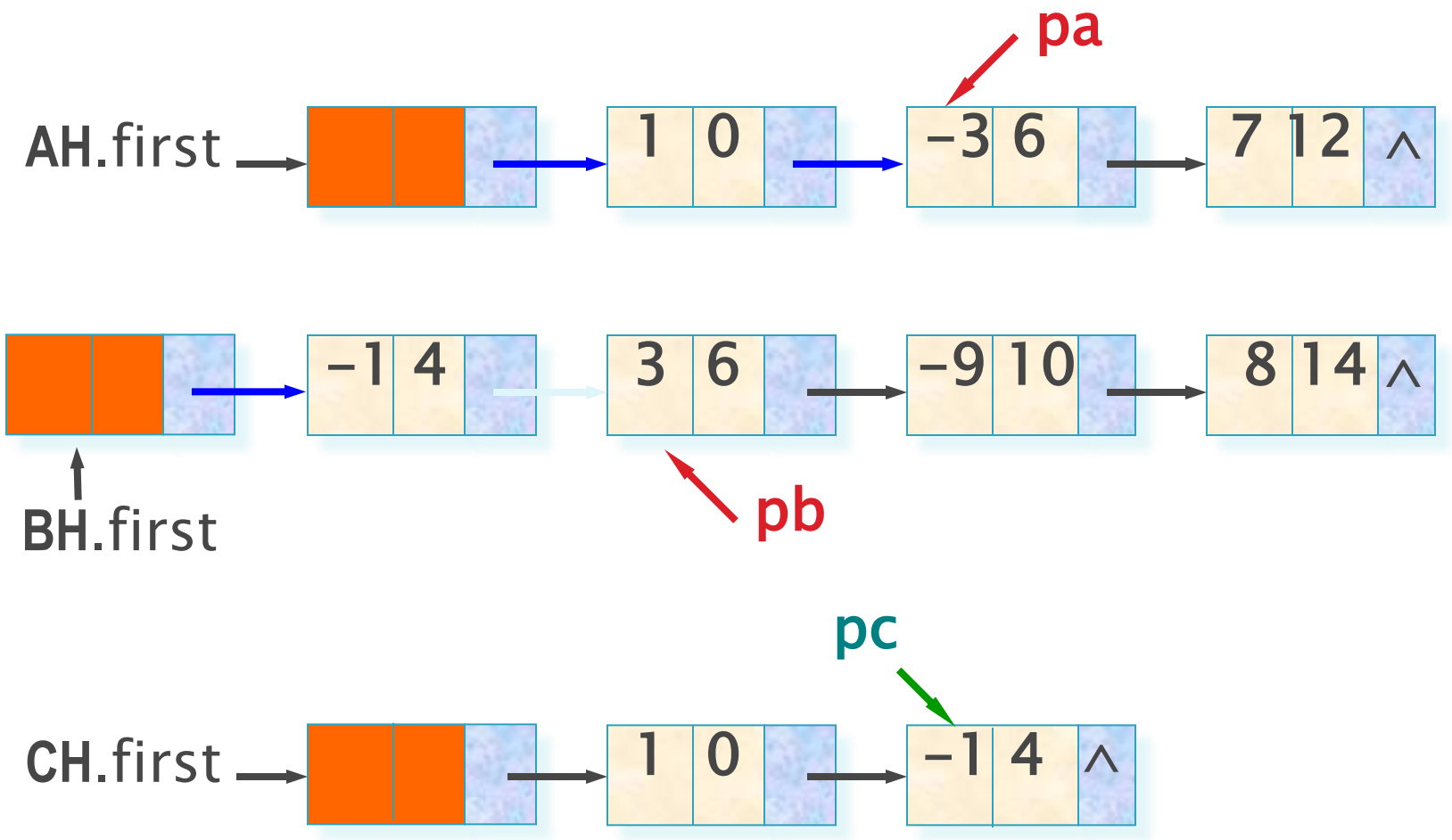
$BH = -x^4 + 3x^6 - 9x^{10} + 8x^{14}$

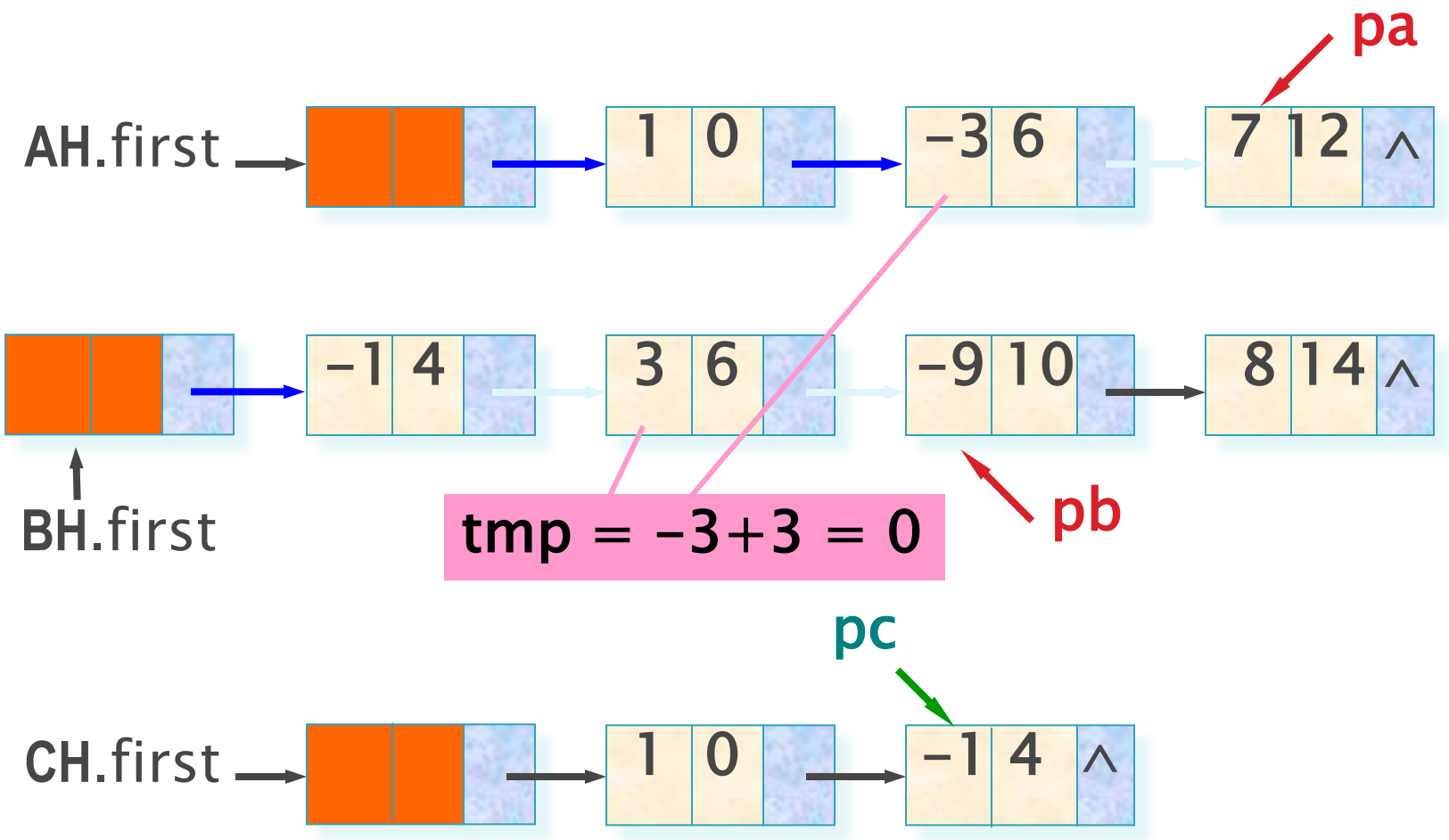
$CH = 1 - x^4 - 9x^{10} + 7x^{12} + 8x^{14}$

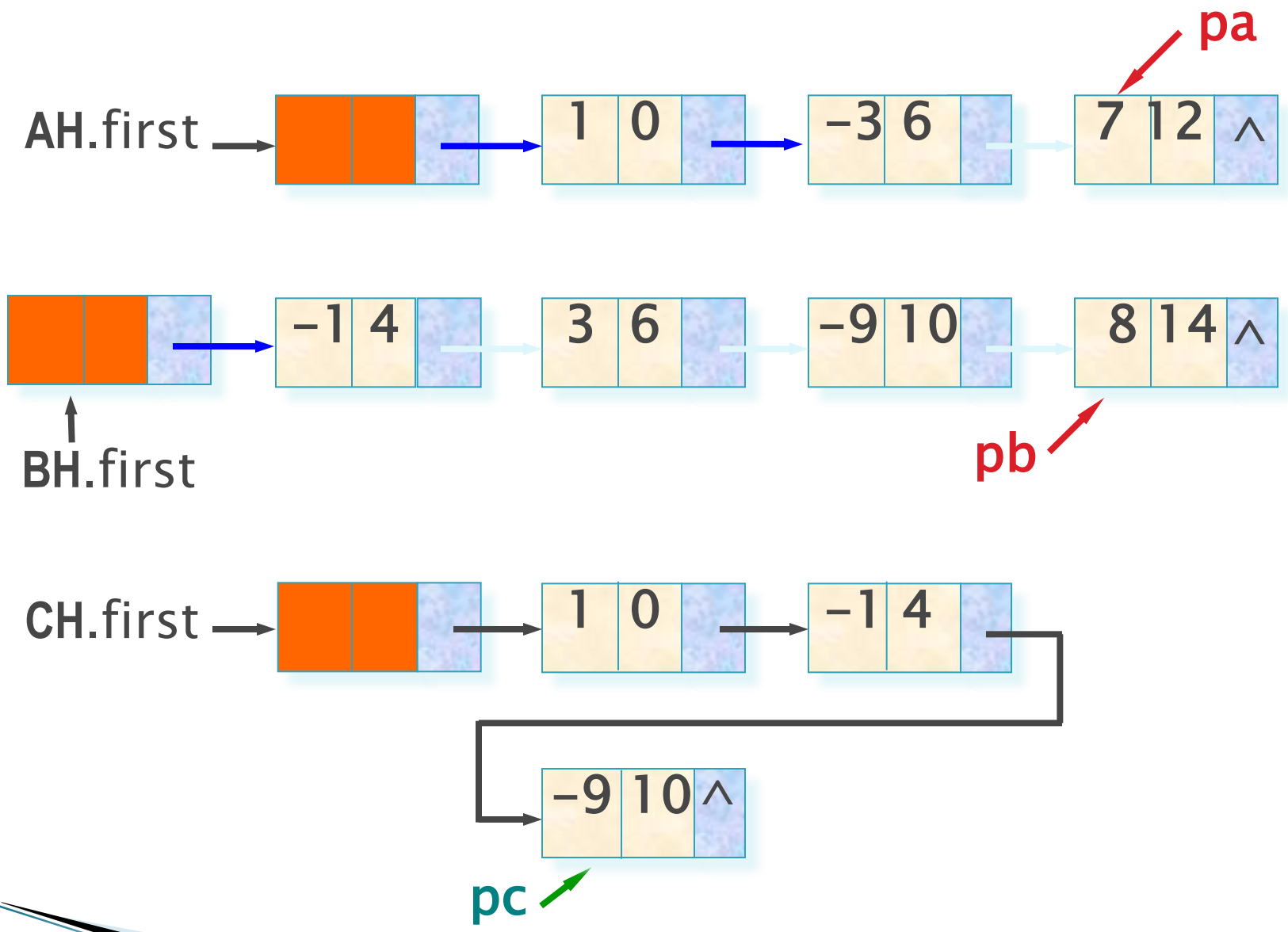


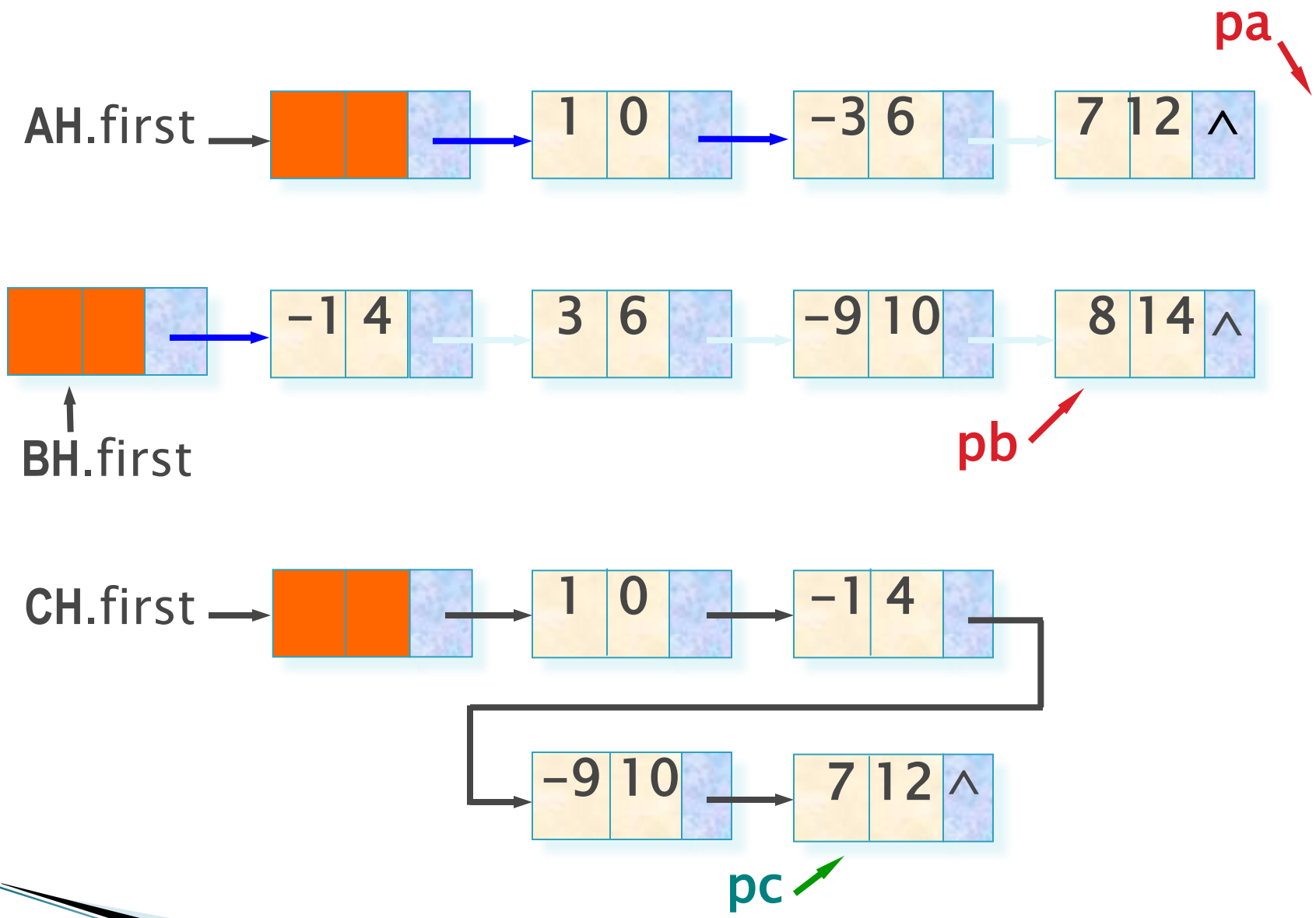

pa
AH.first
1 0
-3 6
7 12 ^
-1 4
3 6
-9 10
8 14 ^
BH.first
pb
pc
CH.first
^

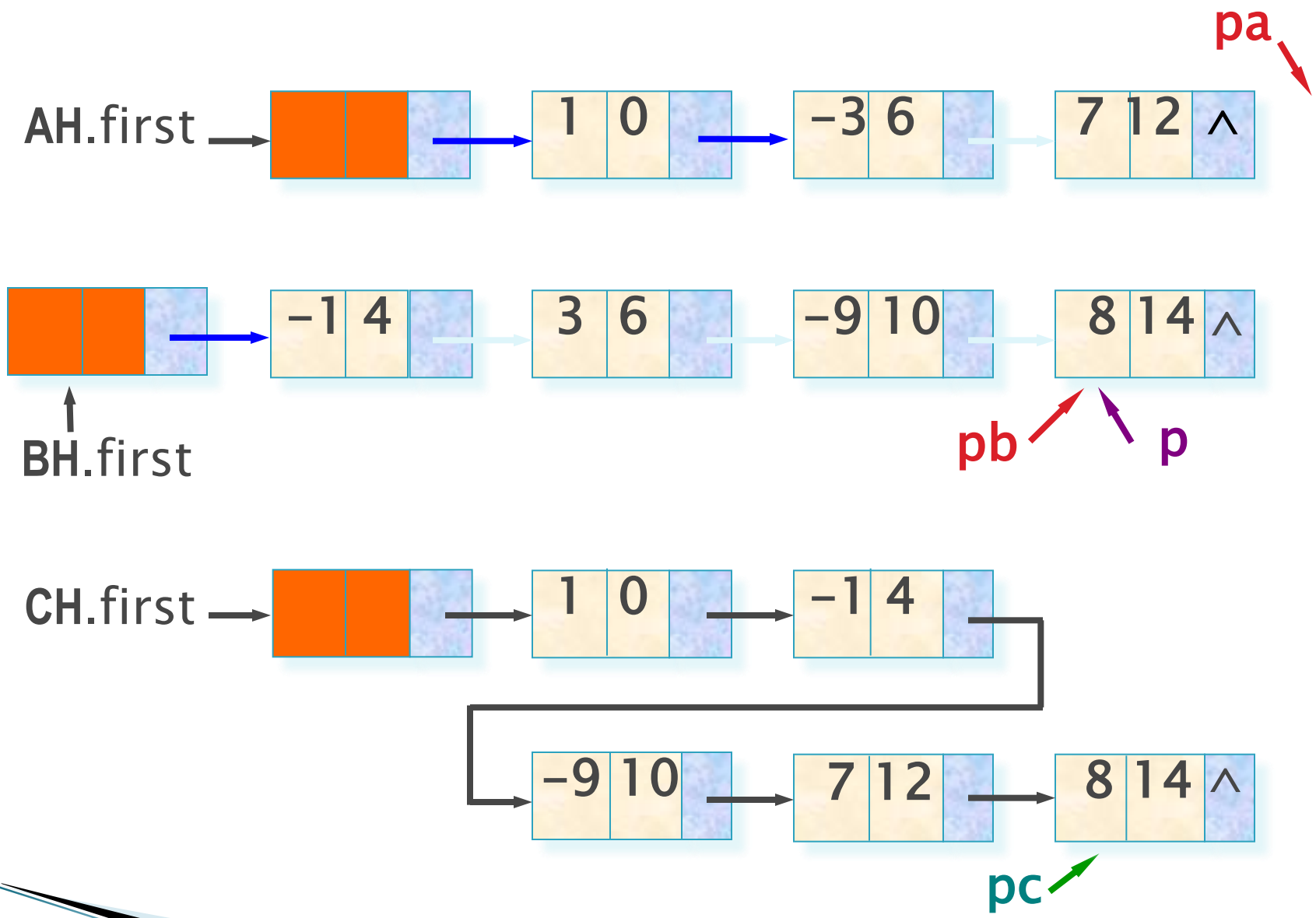

pa
AH.first
1 0
-3 6
7 12 ^
-1 4
3 6
-9 10
8 14 ^
BH.first
pb
pc
CH.first
1 0 ^


pa
AH.first
1 0
-3 6
7 12 ^
-1 4
3 6
-9 10
8 14 ^
BH.first
pb
pc
CH.first
1 0
-1 4 ^


pa
AH.first
1 0
-3 6
7 12 ^
-1 4
3 6
-9 10
8 14 ^
BH.first
tmp = -3+3 = 0
pb
pc
CH.first
1 0
-1 4 ^


pa
AH.first
1 0
-3 6
7 12 ^
-1 4
3 6
-9 10
8 14 ^
BH.first
pb
CH.first
1 0
-1 4
-9 10 ^
pc


pa
AH.first
1 0
-3 6
7 12 ^
-1 4
3 6
-9 10
8 14 ^
BH.first
pb
CH.first
1 0
-1 4
-9 10
7 12 ^
pc


pa
AH.first
1 0
-3 6
7 12 ^
-1 4
3 6
-9 10
8 14 ^
BH.first
pb
p
CH.first
1 0
-1 4
-9 10
7 12
8 14 ^
pc

```
Status AddPolyn ( polynomial &Pc,
                  polynomial &Pa, polynomial &Pb) {   //
利用两个多项式的结点构成“和多项式” Pc = Pa + Pb
                        ... ...
   if (DelAfter(Pa, e1))  a=e1.expn   else  a=MAXE;
   if (DelAfter(Pb, e2))  b=e2.expn   else  b=MAXE;
   while (!(a=MAXE && b=MAXE)) {
                  ... ...
    }
              ... ...
} // AddPolyn
```



```
switch (*cmp(e1, e2)) {
    case -1: {  // 多项式PA中当前结点的指数值小
          InsAfter(Lc, e1);
           if ( DelAfter( Pa, e1) ) a=e1.expn;
          else a = MAXC;    break;  }
    case 0: {   // 两者的指数值相等
        e1.coef= a.coef + b.coef ;
        if ( a.coef != 0.0 ) InsAfter(Pc, e1);
        if ( DelAfter( Pa, e1) ) a=e1.expn;
          else a = MAXC;
         if ( DelAfter( Pb, e2) ) b=e2.expn;
              else b = MAXC;   break;
    }
    case 1: {  //多项式PB中当前结点的指数值小
       break;  }
}
```

一元多项式相加

选择指数较小的一项

两条输入链表按指数严格递增，结点动态分配且互不共享。函数重接原结点，释放相消或合并的项。

```
#include <stdlib.h>

typedef struct TermNode {
    double coef;
    int expn;
    struct TermNode *next;
} TermNode;

TermNode *add_polynomials(TermNode *a, TermNode *b)
{
    TermNode dummy = {0, 0, NULL};
    TermNode *tail = &dummy;
    while (a && b) {
        TermNode *term;
        if (a->expn < b->expn) {
            term = a;
            a = a->next;
        } else if (b->expn < a->expn) {
            term = b;
            b = b->next;  // 接入 B 的较低次项后，推进 B
```

一元多项式相加

合并同次项并接上剩余链表


```
        } else {
            term = a;
            a = a->next;
            term->coef += b->coef;
            TermNode *merged = b;
            b = b->next;
            free(merged);
            if (term->coef == 0.0) {
                free(term);  // 同次项相消，结果不保留零项
                continue;
            }
        }
        tail->next = term;
        tail = term;
    }
    tail->next = a ? a : b;  // 剩余项已经按指数有序
    return dummy.next;
}
```

顺序表和单链表的比较

存储分配方式比较

- 顺序表采用顺序存储结构，即用一段地址**连续**的存储单元**依次**存储线性表的数据元素，数据元素之间的逻辑关系通过**存储位置**来实现。

- 单链表采用链接存储结构，即用一组**任意**的存储单元存放线性表的元素。用**指针**来反映数据元素之间的逻辑关系。

顺序表和单链表的比较

时间性能比较

时间性能是指实现基于某种存储结构的基本操作（即算法）的时间复杂度。

按位查找：

- **顺序表的时间为 $O(1)$，是随机存取；**
- **单链表的时间为 $O(n)$，是顺序存取。**

插入和删除：

- **顺序表需移动表长一半的元素，时间为 $O(n)$；**
- **单链表不需要移动元素，在给出某个合适位置的指针后，插入和删除操作所需的时间仅为 $O(1)$。**

空间性能比较

空间性能是指某种存储结构所占用的存储空间的大小。

定义结点的存储密度：

$$\text{存储密度} = \frac{\text{数据域占用的存储量}}{\text{整个结点占用的存储量}}$$

空间性能比较

结点的存储密度：

- 顺序表中每个结点的存储密度为1（只存储数据元素），没有浪费空间；
- 单链表的每个结点的存储密度<1（包括数据域和指针域），有指针的结构性开销。

整体结构：

- 顺序表需要预分配存储空间，如果预分配得过大，造成浪费，若估计得过小，又将发生上溢；
- 单链表不需要预分配空间，只要有内存空间可以分配，单链表中的元素个数就没有限制。

结论

(1)若线性表需**频繁查找**却很少进行插入和删除操作，或其操作和元素在表中的位置密切相关时，宜采用顺序表作为存储结构；若线性表需**频繁插入和删除（即动态性较强）**时，则宜采用单链表做存储结构。

(2)当线性表中元素**个数变化**较大或者未知时，最好使用单链表实现；而如果用户事先知道线性表的大致长度，使用顺序表的空间效率会更高。

总之，线性表的顺序实现和链表实现各有其优缺点，不能笼统地说哪种实现更好，只能根据实际问题的具体需要，并对各方面的优缺点加以综合平衡，才能最终选定比较适宜的实现方法。

本章小结

1.了解线性表的逻辑结构特性是数据元素之间存在着线性关系，在计算机中表示这种关系的两类不同的存储结构是顺序存储结构和链式存储结构。用前者表示的线性表简称为顺序表，用后者表示的线性表简称为链表。

2.熟练掌握这两类存储结构的描述方法，以及线性表的各种基本操作的实现。

3.能够从时间和空间复杂度的角度综合比较线性表两种存储结构的不同特点及其适用场合。

一元多项式相加

同次项相消

输入 A = 1 + 3x²，B = -3x² + 2x³。链表按指数递增。

预期 结果为 1 + 2x³。指数为 2 的两项相消，结果链中没有这一项。

一条链表为空

输入 A 为空，B = 2x³。

预期 返回 B 的首结点，仍为 2x³。